STA 3402 – Machine Learning & Neural Networks

6 වන දිනයේ දේශන සටහන් (Day 6)

නිර්දේශක පද්ධති – 2 කොටස

(Recommender Systems – Part 2)

Collaborative Filtering • Jaccard සහ Cosine Similarity • Ratings Normalize කිරීම • UV-Decomposition සහ RMSE • Netflix Challenge • R වලින් Recommender එකක් හැදීම

මේ දේශනයේ ආවරණය වන දේ

1. පසුගිය දේශනයේ කෙටි පුනරීක්ෂණයක් (recommender system එකක් කියන්නේ මොකක්ද, content-based සහ collaborative filtering අතර වෙනස, utility matrix එක, computers උදාහරණය).
2. **Collaborative filtering** – user දෙන්නෙක් කොච්චර සමානද කියලා මනින හැටි: **Jaccard distance**, **cosine distance**, **data round කිරීම** සහ **ratings normalize කිරීම**.
3. පන්තියේදී R වලින් කළ වැඩ: utility matrix එකක් normalize කිරීම, binary (0/1) matrix එකක් හැදීම, Jaccard distance function එකක් ලිවීම.
4. **Dimensionality reduction** – **UV-decomposition** සහ **root-mean-square error (RMSE)**.
5. **Netflix Challenge** (ඩොලර් 1,000,000 ත්‍යාගය).
6. MovieLens 100K data set එක පාවිච්චි කරලා **R වලින් මුල සිටම** ඇත්ත recommender system එකක් හැදීම.
7. Homework, විභාග ඉඟි සහ නිවේදන.

විභාග ඉඟිය (Exam tip)

දේශකතුමා පැහැදිලිව කිව්වා: **Jaccard similarity/distance, cosine similarity/distance, rounding සහ normalizing අවසාන විභාගයට** එනවා – **අතින් (manually)** කරන්නත්, **R වලින්** කරන්නත්.

UV-decomposition / RMSE එක පියවරෙන් පියවර optimize කරන කොටස විභාගයට එන්නේ නැහැ – ඒක ඔයාගේ දැනුමට විතරයි.

Lab-based විභාගයේදී data set එකක් දීලා **recommender system එකක් හදන්න** කියන්න පුළුවන්.

මේ සටහන් හදන්න පාවිච්චි කළේ: 6 වන දින දේශන පටිගත කිරීම, පන්තියේ R code එක, whiteboard පිටු (Day 1 සහ Day 2), සහ පෙළපොතේ *"Recommendation Systems"* පරිච්ඡේදය (Chapter 9, Mining of Massive Datasets). පහත සියලු ගණනය කිරීම් R වලින් පරීක්ෂා කළා. Technical වචන වරහන් තුළ ඉංග්‍රීසියෙන් දාලා තියෙනවා – විභාගයේදී ඒ වචන ඔයාට හඳුනාගන්න පුළුවන් වෙන්න.

1. පසුගිය දේශනයේ කෙටි පුනරීක්ෂණය

දේශනය පටන් ගත්තේ පසුගිය වතාවේ ඉගෙනගත් දේවල් ඉක්මනින් මතක් කරලා, එතැන් සිට ඉදිරියට යන්න.

1.1 Recommender system එකක් කියන්නේ මොකක්ද?

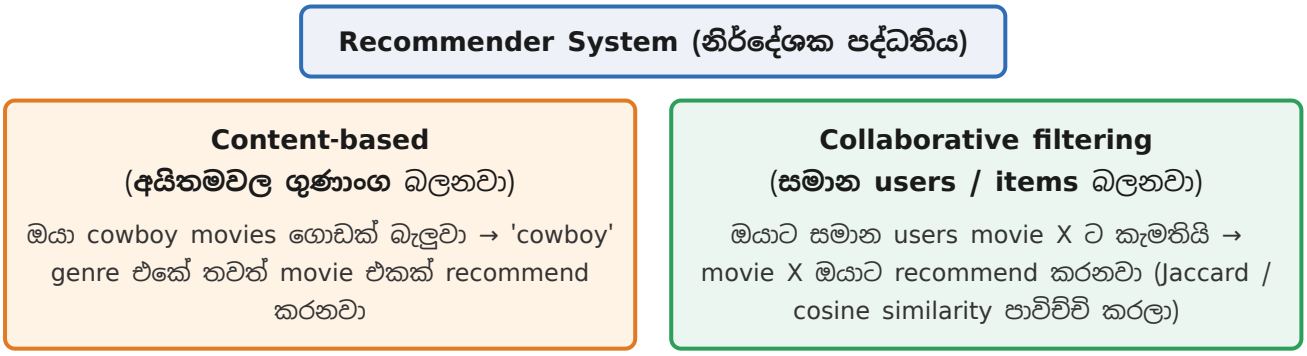
Recommender (recommendation) system එකක් කියන්නේ user කෙනෙකුට **යමක් නිර්දේශ කරන (recommend කරන)** පද්ධතියක්. ඒකෙන් user කැමති වෙන්නේ මොනවටද කියලා predict කරලා, ඒවා යෝජනා කරනවා. පන්තියේදී දුන් උත්තර (Day 1 whiteboard):

- **Spotify** ඔයාට ගැලපෙන සින්දු හෝ playlists යෝජනා කරනවා.
- ඔයා online **camera එකක් මිලදී ගත්තොත්**, system එක **camera accessories** ත් යෝජනා කරනවා.
- Statistical method එකක් තෝරගැනීමත් එක්තරා recommendation එකක්: උදා: data වල **outliers** තියෙනකොට mean එක වෙනුවට **median** එක නිර්දේශ කරනවා.
- පෙළපොතේ අනෙක් උදාහරණ: online පුවත්පත් කියවන අයට ඔවුන්ගේ උනන්දුව අනුව **news articles** දීම; online shop එකක customers ලට ඔවුන්ගේ කලින් මිලදී ගැනීම් හෝ searches අනුව බඩු යෝජනා කිරීම.

Day 1 හි දෙවන ප්‍රශ්නය: **Netflix Challenge එක කියන්නේ මොකක්ද?** – Netflix එකේම recommendation system එක දියුණු කරන්න තිබුණු තරඟයක්. ඒක 5 වන කොටසේ විස්තර කරනවා.

1.2 Recommender systems වල ප්‍රධාන කාණ්ඩ දෙක

Recommendation systems විවිධ තාක්ෂණ පාවිච්චි කරනවා, නමුත් අපිට ඒවා **ප්‍රධාන කාණ්ඩ දෙකකට** බෙදන්න පුළුවන්. **මෙතැන් සිට ඔයා recommender system එකක් දැක්කොත්, පළමුව බලන්න ඕනේ ඒක content-based ද collaborative filtering ද කියලා** – මේ topic එක ඉගෙනගන්නේ එහෙමයි.



- **Content-based systems** – recommend කරන **අයිතමවල ගුණාංග (content)** පරීක්ෂා කරනවා. උදා: Netflix user කෙනෙක් cowboy movies ගොඩක් බලලා නම්, database එකේ “cowboy” genre එකට අයිති movie එකක් recommend කරනවා – ඔහු කලින් බලපු content එක මත පදනම්ව.
- **Collaborative filtering systems** – **users සහ/හෝ items අතර similarity මිනුම්** මත පදනම්ව අයිතම recommend කරනවා. User කෙනෙකුට recommend කරන්නේ ඔහුට සමාන users කැමති අයිතම. (පසුගිය වතාවේ cosine similarity වගේ මිනුම් ඉගෙනගත්තා.)

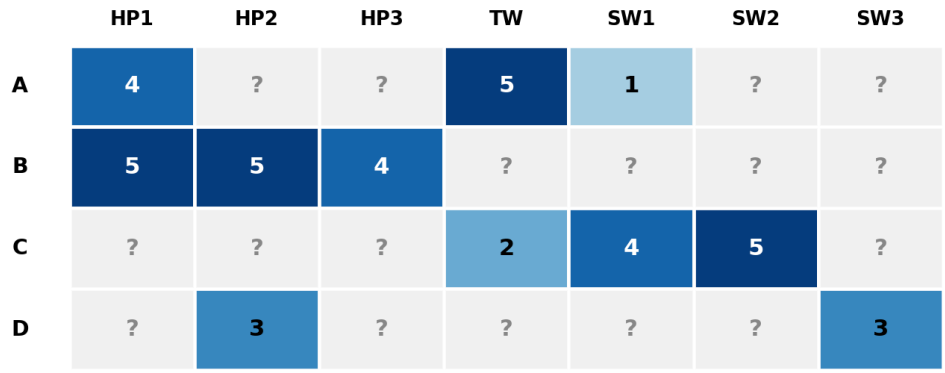
1.3 Recommender systems සඳහා model එකක්: utility matrix එක

Data ටික අර්ථවත් විදිහට තියාගන්න ඕනේ. ඒකට සාමාන්‍ය ක්‍රමය තමයි **utility matrix** එක. Recommender system එකක දේවල් වර්ග දෙකක් තියෙනවා: **users** (පරිශීලකයන්) සහ **items** (අයිතම). Utility matrix එකෙන් එක් එක් user–item යුගලයට, ඒ user ඒ item එකට කොච්චර කැමතිද කියන අගයක් දෙනවා (උදා: තරු 1–5 rating එකක්).

උදාහරණය (Figure 9.1 / 9.4): users 4 දෙනෙක් (A, B, C, D) සහ movies 7ක් – Harry Potter 1, 2, 3 (HP1, HP2, HP3), Twilight (TW), Star Wars 1, 2, 3 (SW1, SW2, SW3). ඉතින් මේක **4 × 7 matrix** එකක් (rows 4, columns 7). Ratings 1–5 පරිමාණයෙන් (5 = ඉහළම). **හිස් තැනක් (blank)** කියන්නේ user ඒ movie එක **rate කරලා නැහැ** කියන එක.

	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	4			5	1		
B	5	5	4				
C				2	4	5	
D		3					3

Utility matrix: users (rows) x movies (columns). '?' = blank (not rated)



Utility matrix එක: users (rows) × movies (columns). '?' = blank (rate කරලා නැහැ)

- පන්තියේදී දේශකතුමා ශිෂ්‍යයන්ට මෑතකදී බලපු movie එකක් (උදා: Spider-Man) 1 සිට 5 දක්වා rate කරන්න කිව්වා. ඒක ඔයාගේ රුචිය අනුව වෙනස් වෙනවා: romantic movies වලට කැමති නම් 3 හෝ 4 දෙයි; sci-fi වලට කැමති නම් 4 හෝ 5 දෙයි.
- බොහෝ user–movie යුගල **හිස්**. ප්‍රායෝගිකව matrix එක මීටත් වඩා **sparse** – “sparse” කියන්නේ **blanks (හෝ zeros) ගොඩක් තියෙනවා** කියන එක, මොකද එක් user කෙනෙක් rate කරන්නේ සියලු movies වලින් ඉතා කුඩා කොටසක් විතරයි.
- **Recommender system එකක අරමුණ utility matrix එකේ blanks පුරවන (predict කරන) එකයි.** උදා: A, SW2 ට කැමති වෙයිද? A, SW1 බැලුවා නමුත් **කැමති වුණේ නැහැ** (rating 1), HP1 සහ TW ට කැමතියි. එහෙනම් A, Star Wars 2 ට කැමති වෙයිද? Recommender system එකක් උත්තර දෙන්නේ මේ වගේ ප්‍රශ්නවලට.
- බොහෝ applications වලදී හැම blank එකක්ම predict කරන්න ඕනේ නැහැ – එක් එක් row එකේ **ඉහළ** rating එකක් ලැබෙන්න ඉඩ ඇති අයිතම කිහිපයක් හොයාගෙන ඒවා recommend කළොත් ඇති.

පසුගිය වතාවේ **long tail** එක ගැනත් කතා කළා (online shops වලට අයිතම මිලියන ගණනක් දෙන්න පුළුවන්, නමුත් භෞතික “brick-and-mortar” කඩවලට පෙන්වන්න පුළුවන් වඩාත් ජනප්‍රිය අයිතම ටික විතරයි, ඒ නිසා online shops **අනිවාර්යයෙන්ම** recommend කරන්න ඕනේ) සහ

product recommendations, movie recommendations, news articles වගේ **applications** ගැනත් කතා කළා.

1.4 Content-based පුනරීක්ෂණය: අයිතම vectors විදිහට

Content-based recommendation එක අවධානය යොමු කරන්නේ **අයිතමවල ගුණාංග** වලට; collaborative filtering අවධානය යොමු කරන්නේ **users සහ items අතර සම්බන්ධතා** වලට (එනම් similarity).

Example 9.2 (පසුගිය දේශනයෙන්). Movies වල features විදිහට තියෙන්නේ **නළුවන්ගේ කට්ටලය (set of actors)** සහ **සාමාන්‍ය rating එක (average rating)** විතරයි කියලා හිතමු. Movies දෙකක නළුවන් පස් දෙනා බැගින් ඉන්නවා; ඒ අතරින් දෙදෙනෙක් movies දෙකේම ඉන්නවා. එක movie එකක average rating එක 3, අනෙකේ 4. Vectors මෙහෙමයි:

	act 1	act 2	act 3	act 4	act 5	act 6	act 7	act 8	avg rating
Movie 1	0	1	1	0	1	1	0	1	3α
Movie 2	1	1	0	1	0	1	1	0	4α

න්‍යායිකව තවත් නළුවන් අනන්ත ගණනක් ඉන්නවා (vectors දෙකේම 0 තියෙන components), නමුත් vectors දෙකේම 0 තියෙන components වලින් cosine distance එකට **බලපෑමක් නැහැ**. α කියන්නේ rating එක සඳහා **scaling factor** එකක්. කෝණයේ cosine අගය:

$$\cos\theta = (2 + 12\alpha^2) \,/\, \sqrt{(25 + 125\alpha^2 + 144\alpha^4)}$$

α = 1 → 0.816; α = 2 → 0.940 (වඩා සමාන වගේ පේනවා); α = 1/2 → 0.619 (ගොඩක් වෙනස් වගේ පේනවා). ඉතින් **scaling factor එක තෝරගන්න විදිහ අනුව අයිතම කොච්චර සමානද කියන එක වෙනස් වෙනවා**.

1.5 Computers උදාහරණය – scaling ඕනේ ඇයි?

Exercise 9.2.1. Computers තුනක් A, B සහ C වලට මේ numerical features තියෙනවා:

Feature	A	B	C
Processor speed	3.06	2.68	2.92
Disk size	500	320	640
Main-memory size	6	4	6

එක් එක් computer එක vector එකක් විදිහට හිතන්න පුළුවන්, උදා: A = [3.06, 500, 6]. ඕනෑම vectors දෙකක් අතර cosine distance එක ගණනය කරන්න පුළුවන්, නමුත් **features එකම පරිමාණයේ (scale) නැහැ**: disk size සිය ගණන්වල, processor speed 3 පමණ. Scale නොකළොත් **disk size එක ගණනය කිරීම ආධිපත්‍යය කරනවා (dominate)**, අනෙක් features වල වෙනස්කම් පේන්නේ නැතිව යනවා. ඒ නිසා processor speed ට scale factor 1, disk size ට α, main memory ට β පාවිච්චි කරලා, α සහ β ඇසුරින් cosines ගණනය කරනවා (පසුව උදා: α = β = 1 සඳහා).

සටහන

α = β = 1 (scaling නැතිව) පාවිච්චි කළොත් cosines තුනම ආසන්න වශයෙන් 0.99999 (කෝණ 0.3° ට අඩුයි), එනම් computers තුනම බොහෝ දුරට සමාන වගේ පේනවා – ඒ disk size එක dominate කරන නිසා විතරයි. Scaling ඕනේ ඇයි කියන්නේ හරියටම මේකයි.

පසුගිය දේශනයේ homework (Day 1 whiteboard)

Exercise 9.2.2. Scale කරන්න තවත් ක්‍රමයක් තමයි vectors **normalize** කිරීම: එක් එක් component එකේ (feature) සාමාන්‍යය ගණනය කරලා, එය හැම vector එකකම ඒ component එකේ අගයෙන් අඩු කරනවා. (a) Computers තුනේ vectors normalize කරන්න.

Exercise 9.2.3. User කෙනෙක් computers මෙහෙම rate කළා: A = තරු 4, B = තරු 2, C = තරු 5. (a) මේ user ගේ ratings normalize කරන්න. (b) Processor speed, disk size සහ main memory size components සහිතව user profile එකක් ගණනය කරන්න.

දේශකතුමා normalized අගයන් සහ part (b) උත්තරය එවන්න කිව්වා: *“මේවා final exam එකටත් එනවා, ඒ නිසා මේක කරන හැටි දැනගන්න ඕනේ.”*

විසඳූ උත්තර (ඔයාගේ වැඩ පරීක්ෂා කරන්න උදව්වට එකතු කළා)

9.2.2 (a) සාමාන්‍යයන්: speed = (3.06 + 2.68 + 2.92)/3 = 2.887; disk = (500 + 320 + 640)/3 = 486.67; memory = (6 + 4 + 6)/3 = 5.333.

Normalized: A = [3.06 − 2.887, 500 − 486.67, 6 − 5.333] = **[0.173, 13.33, 0.667]**; B = **[−0.207, −166.67, −1.333]**; C = **[0.033, 153.33, 0.667]**.

9.2.3 (a) User ගේ සාමාන්‍ය rating එක = (4 + 2 + 5)/3 = 11/3. Normalized ratings: A = 4 − 11/3 = **1/3**, B = 2 − 11/3 = **−5/3**, C = 5 − 11/3 = **4/3**.

9.2.3 (b) එක් එක් computer එකේ features, normalized rating එකෙන් ගුණ කරලා (weight කරලා) සාමාන්‍යය ගන්න:

speed = (1/3·3.06 − 5/3·2.68 + 4/3·2.92)/3 = **0.149**; disk = (1/3·500 − 5/3·320 + 4/3·640)/3 = **162.2**; memory = (1/3·6 − 5/3·4 + 4/3·6)/3 = **1.111**. ඉතින් user profile ≈ [0.149, 162.2, 1.111] – මේ user විශාල disks සහ වැඩි memory වලට කැමතියි.

2. Collaborative filtering

කලින් කොටස **content-based** filtering. දැන් අපි **සැලකිය යුතු ලෙස වෙනස් ක්‍රමයක්** ගන්නවා.

- Similarity තීරණය කරන්න **අයිතමවල features** පාවිච්චි කරනවා වෙනුවට, අයිතම දෙකක් සඳහා **users දුන් ratings වල similarity** එක ගැන අවධානය යොමු කරනවා.
- අයිතමයක item-profile vector එක වෙනුවට, utility matrix එකේ **ඒ අයිතමයේ column එක** පාවිච්චි කරනවා.
- Users ලට profile vector එකක් හදනවා වෙනුවට, එක් එක් user utility matrix එකේ **ඔහුගේ row එකෙන්** නිරූපණය කරනවා.
- Users දෙන්නෙක් **සමානයි** කියන්නේ, **Jaccard distance** හෝ **cosine distance** වගේ distance මිනුමකට අනුව ඔවුන්ගේ vectors (rows) **ළඟින්** තියෙනවා නම්.
- User U ට recommendation එකක් දෙන්නේ **U ට වඩාත්ම සමාන users** හොයාගෙන, **ඒ users කැමති අයිතම** recommend කරලා.

> **ප්‍රධාන අදහස**
>
> **Collaborative filtering = සමාන users හඳුනාගෙන සමාන users කැමති දේ recommend කරන** ක්‍රියාවලිය.

2.1 Similarity මැනීම

පළමු ප්‍රශ්නය: utility matrix එකේ **users (rows) හෝ items (columns) වල similarity එක මනින්නේ කොහොමද?**

අපිට **continuous** variables දෙකක් X සහ Y (උදා: උස සහ බර) තිබුණා නම්, **Pearson correlation** එකෙන් similarity මනින්න තිබුණා. නමුත් මෙතැන තියෙන්නේ blanks ගොඩක් සහිත ratings, ඒ නිසා වෙනත් මිනුම් බලනවා.

අපි ආයෙත් Figure 9.4 පාවිච්චි කරනවා (Figure 9.1 ම තමයි). විශ්වාසදායක නිගමනවලට එන්න මේ data ඉතා කුඩායි, නමුත් කුඩා නිසාම එක් එක් distance මිනුමේ **අඩුපාඩු (pitfalls)** පැහැදිලිව පේනවා.

	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	4			5	1		
B	5	5	4				
C				2	4	5	
D		3					3

විශේෂයෙන් users A සහ C බලන්න. ඔවුන් දෙදෙනාම rate කරපු **movies දෙකක්** තියෙනවා: **TW** සහ **SW1** (highlight කරලා). නමුත් ඔවුන්ගේ අදහස් **බොහෝ දුරට සම්පූර්ණයෙන්ම ප්‍රතිවිරුද්ධයි**:

- User A, Twilight ට **5** දෙනවා (ගොඩක් කැමතියි), Star Wars 1 ට **1** ක් විතරයි.
- User C, Twilight ට **2** දෙනවා, Twilight ට වඩා Star Wars 1 ට (**4**) කැමතියි.

ඉතින් **හොඳ distance මිනුමක් A සහ C එකිනෙකාගෙන් තරමක් ඈතින් තියන්න ඕනේ**. හොඳ metric එකක් මේ අසමානකම් අල්ලගන්න ඕනේ.

පන්තියේ කියපු ඇත්ත ජීවිත උදාහරණය: dating apps (උදා: Tinder)

මේ similarity ක්‍රම ප්‍රායෝගිකව පාවිච්චි වෙන හැටි දේශකතුමා පැහැදිලි කළා, උදාහරණයක් විදිහට **Tinder** වගේ dating apps. App එක මුහුණක් සහ මූලික විස්තර කිහිපයක් පෙන්වනවා; ඔයා කැමති නම් **දකුණට swipe** කරනවා, අකමැති නම් **වමට swipe** කරනවා. කිලෝමීටර් කීයක් ඇතුළතද (radius) කියලත් set කරන්න පුළුවන්.

ටික කාලයකට පස්සේ algorithm එක **similarities** (ඔයාගේ වයස, කැමැත්ත, swipes) බලලා ඔයාට **recommendations** දෙනවා. සමහර රටවල අය ඊට පස්සේ කෝපි එකකට හම්බවෙලා ඉදිරියට යනවද නැද්ද කියලා තීරණය කරනවා. ප්‍රධාන කාරණය: **algorithm එක බලන්නේ users අතර similarities** – අපි මෙතැන කරන්නෙත් හරියටම ඒකමයි.

2.2 Jaccard distance

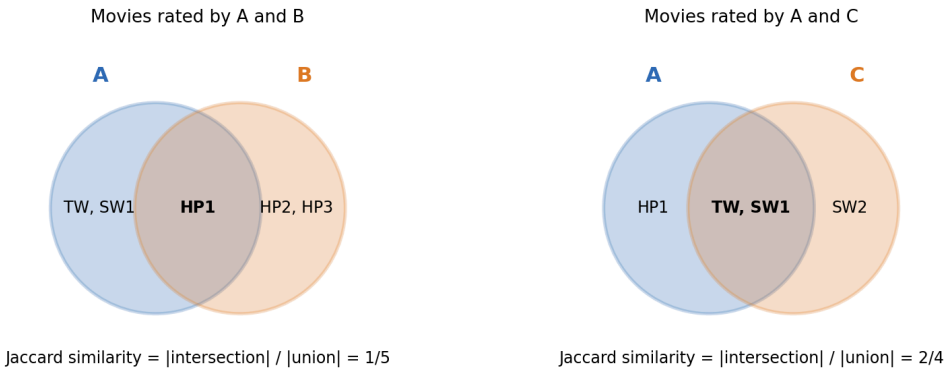
Jaccard වලදී අපි **අගයන් (ඇත්ත ratings) නොසලකා හරිනවා, rate කරපු අයිතම කට්ටල (sets)** ගැන විතරක් අවධානය යොමු කරනවා.

$$\textbf{Jaccard similarity } J(A,B) = |A \cap B| \,/\, |A \cup B| \qquad \textbf{Jaccard distance} = 1 - J(A,B)$$

($\cap$ = intersection (ඡේදනය) = කට්ටල **දෙකේම** තියෙන අයිතම; $\cup$ = union (සමිතිය) = **අවම වශයෙන් එක** කට්ටලයක හෝ තියෙන අයිතම.)

- Utility matrix එකේ තිබුණේ **මිලදී ගැනීම්** (ගත්තා / ගත්තේ නැහැ) විතරක් නම්, Jaccard තෝරගන්න හොඳ මිනුමක්.
- නමුත් utilities **විස්තරාත්මක ratings** නම්, Jaccard distance එකෙන් **වැදගත් තොරතුරු නැති වෙනවා** (rating එක 1 ද 5 ද කියලා ඒක ගණන් ගන්නේ නැහැ).

Example 9.7 – පියවරෙන් පියවර



1. A rate කරපු movies = {HP1, TW, SW1}. B rate කරපු movies = {HP1, HP2, HP3}.
2. Intersection A $\cap$ B = {HP1} → ප්‍රමාණය **1**. Union A $\cup$ B = {HP1, HP2, HP3, TW, SW1} → ප්‍රමාණය **5**.
3. Jaccard similarity (A, B) = 1/5 = 0.2, ඒ නිසා Jaccard distance = 1 − 1/5 = **4/5** (ගොඩක් ඈතයි).
4. C rate කරපු movies = {TW, SW1, SW2}. A $\cap$ C = {TW, SW1} → 2. A $\cup$ C = {HP1, TW, SW1, SW2} → 4.
5. Jaccard similarity (A, C) = 2/4 = 0.5, ඒ නිසා Jaccard distance = 1 − 1/2 = **1/2** (අඩක් දුරින්).

ඉතින් Jaccard කියන්නේ: similarity(A, C) = 0.5 > similarity(A, B) = 0.2, එනම් **A, B ට වඩා C ට ළඟින් ඉන්නවා වගේ පේනවා**.

සටහන

නමුත් මේ නිගමනය **බුද්ධියට එකඟව වැරදියි (intuitively wrong)**! A සහ C දෙදෙනාම බලපු movies දෙක ගැන **එකඟ නැහැ**, නමුත් A සහ B ඔවුන්ට පොදු එකම movie එකට **දෙදෙනාම කැමතියි** (HP1: A දුන්නේ 4, B දුන්නේ 5). Jaccard මෙතැනදී අසාර්ථක වෙන්නේ rating අගයන් නොසලකා හරින නිසා.

විභාග ඉඟිය (Exam tip)

ඕනෑම users යුගලයක් සඳහා Jaccard similarity සහ Jaccard distance අතින් ගණනය කරන්න පුළුවන් වෙන්න ඕනේ.

2.3 Cosine distance (cosine similarity)

Cosine වලදී අපි rating අගයන් **සැලකිල්ලට ගන්නවා**. **Blanks 0 විදිහට** සලකනවා.

සටහන

Blanks 0 විදිහට සැලකීම **ප්‍රශ්නසහගතයි**: එයින් "rate කරලා නැහැ" කියන එක, movie එකට කැමති වීමට වඩා **අකමැති වීමට සමාන** විදිහට සලකනවා.

$$\cos\theta = (x \cdot y) / (\|x\| \times \|y\|) = \Sigma x_i y_i / (\sqrt{\Sigma x_i^2} \times \sqrt{\Sigma y_i^2})$$

උඩ කොටස (numerator) **dot product** එක – users **දෙදෙනාම** rate කරපු movies වල ratings ගුණ කරලා එකතු කරන්න (blanks 0 නිසා ඒවායින් කිසිම දායකත්වයක් නැහැ). යට කොටස (denominator) එක් එක් vector එකේ දිග – එක් එක් user ගේ **සියලු** ratings වර්ග කරලා, එකතු කරලා, වර්ගමූලය ගන්න.

Example 9.8 – A සහ B

පොදු movie එක: HP1 (A = 4, B = 5). A ගේ ratings: 4, 5, 1. B ගේ ratings: 5, 5, 4.

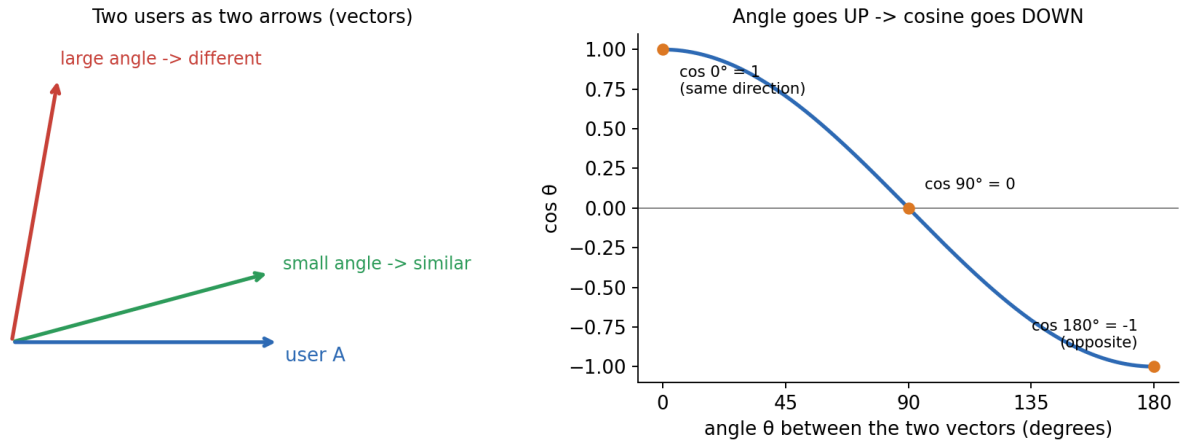
$$\cos(A, B) = (4 \times 5) / (\sqrt{(4^2 + 5^2 + 1^2)} \times \sqrt{(5^2 + 5^2 + 4^2)}) = 20 / (\sqrt{42} \times \sqrt{66}) = 20 / 52.65 = 0.380$$

පන්තියේ අභ්‍යාසය – A සහ C (දේශකතුමා පන්තියට කරන්න කිව්වා)

පොදු movies: TW (A = 5, C = 2) සහ SW1 (A = 1, C = 4). පොදු movies දෙකක් තියෙනකොට ගුණිත **එකතු** කරනවා. C ගේ ratings: 2, 4, 5.

$$\cos(A, C) = (5 \times 2 + 1 \times 4) / (\sqrt{(4^2 + 5^2 + 1^2)} \times \sqrt{(2^2 + 4^2 + 5^2)}) = 14 / (\sqrt{42} \times \sqrt{45}) = 14 / 43.47 = 0.322$$

Cosine අගය තේරුම් ගන්නේ කොහොමද



වම: users දෙදෙනෙක් ඊතල (vectors) දෙකක් විදිහට. දකුණ: කෝණය වැඩි වෙනකොට cosine අගය අඩු වෙනවා.

- Trigonometry වලින් cos θ මතක් කරගන්න: සෘජුකෝණී ත්‍රිකෝණයක cos θ = බද්ධ පාදය / කර්ණය. cos 0° = 1, cos 90° = 0.
- **කෝණය වැඩි වෙනකොට cosine අගය අඩු වෙනවා**.
- ඉතින් **cosine අගය විශාල → කෝණය කුඩා → distance එක කුඩා → users ළඟයි (වඩා සමානයි)**.
- මෙතැන cos(A, B) = 0.380 > cos(A, C) = 0.322, ඒ නිසා cosine කියන්නේ **A, C ට වඩා B ට ටිකක් ළඟයි** කියලා (Jaccard ට වඩා හොඳයි, නමුත් වෙනස කුඩායි).

සටහන

පටිගත කිරීමේදී දේශකතුමා එක් වතාවක් “small angle means larger distance” කිව්වා – ඒක කටින් වැරදුණු දෙයක්. නිවැරදි නීතිය (පෙළපොතේ තියෙන විදිහට): **කෝණය කුඩා = cosine විශාල = distance කුඩා = සමානයි**.

පන්තියේ කියපු ඇත්ත කතාව: telecom consultancy එකක්

දේශකතුමා කලින් telecom සමාගමකට (Dialog වගේ) consultancy කළා. ඔවුන්ට මාසයකට records මිලියන 10ක් විතර තිබුණා – එක් එක් අද්විතීය user (උදා: 07x-xxxxxxx) සහ ඔවුන් පාවිච්චි කරපු prepaid packages (99, 169, ...). සමහරු packages මාරු කරනවා, සමහරු එකම එකේ ඉන්නවා. ඔවුන්ට users අතර **cosine similarity** එක ඕනේ වුණා.

තමන්ගේම laptops වල (Google Colab / Kaggle නැතිව) **R වලට පැය 6–8ක් විතර** ගියා, නමුත් **Python විනාඩි 30කින් විතර** ඒක කළා. මේ වගේ වැඩවලට Python ගොඩක් වේගවත්.

අපි cosine similarity ගැන lab sessions කරනවා, ඉක්මනින්ම ඔයාලා recommender systems හදනවා.

2.4 Data round කිරීම

User කෙනෙක් **ඉහළින්** rate කරපු movies සහ **පහළින්** rate කරපු movies අතර තියෙන බොරු similarity එක ratings **round කිරීමෙන්** ඉවත් කරන්න උත්සාහ කරන්න පුළුවන්. Ratings 1–5 සංඛ්‍යා විදිහට සලකනවා වෙනුවට 0/1 බවට හරවනවා:

- Ratings **3, 4, 5 → 1** (කැමතියි).
- Ratings **1, 2 → rate නොකළා විදිහට (blank)**.

	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	1			1			
B	1	1	1				
C					1	1	
D		1					1

Figure 9.5: ratings 3, 4, 5 වෙනුවට 1; ratings 1 සහ 2 ඉවත් කළා.

1. A = {HP1, TW}, B = {HP1, HP2, HP3}. A ∩ B = {HP1} → 1; A ∪ B = {HP1, HP2, HP3, TW} → 4. Jaccard similarity = 1/4, ඒ නිසා **Jaccard distance (A, B) = 3/4**.
2. A = {HP1, TW}, C = {SW1, SW2}. **Intersection එකක් නැහැ** → 0; union = 4. Jaccard similarity = 0/4 = 0, ඒ නිසා **Jaccard distance (A, C) = 1 − 0 = 1** (පුළුවන් උපරිම දුර).

දැන් **B ට වඩා C, A ගෙන් ඈතයි** – ඒක **බුද්ධියට එකඟව නිවැරදියි**. Figure 9.5 ට cosine distance යෙදුවත් එකම නිගමනයට එනවා.

2.5 Ratings normalize කිරීම

Figure 9.4 (round කරන්න කලින්) සහ Figure 9.6 (normalize කළ) බලන්න. දේශකතුමා ඇහුවා: *"Matrices බලලා, normalized අගයන් ගන්න පාවිච්චි කරපු සූත්‍රය මොකක්ද කියලා හොයන්න."*

"Normalize" කියන්නේ මොකක්ද?

Computers උදාහරණයේ features ගොඩක් වෙනස් පරිමාණවල තිබුණා (memory ≈ 500, speed ≈ 3), ඒ නිසා කෙලින්ම සංසන්දනය කිරීම හොඳ නැහැ – ඒකයි normalize කරන්නේ. බොහෝ විට normalize කිරීම කියන්නේ අගයන් සංසන්දනය කළ හැකි වෙන්න **යම් සාධකයකින් බෙදීම** (උදා: weights $w_1 \times 5 + w_2 \times 3 + w_3 \times 4 = 1$).

නමුත් Figure 9.6 හි user A ගේ normalized අගයන් එකතු කළොත්: 2/3 + 5/3 − 7/3 = **0**. ඉතින් මෙතැන normalized අගයන්ගේ **එකතුව බිංදුවයි**. ඒක වෙන්න නම් ධන සහ ඍණ අගයන් දෙකම තියෙන්න ඕනේ – **ඉහළම ratings ධන** වෙනවා, **කුඩාම ratings ඍණ** වෙනවා.

දේශකතුමා "normalization in collaborative filtering" කියලා search කරන්න කිව්වා. ශිෂ්‍යයෙක්ගේ උත්තරය: **"එක් එක් row එකේ සාමාන්‍යය, ඒ row එකේ එක් එක් අගයෙන් අඩු කරන්න"** – ඒක නිවැරදියි!

ප්‍රධාන අදහස

Collaborative filtering වල normalize කිරීම: එක් එක් rating එකෙන් **ඒ user ගේ සාමාන්‍ය rating එක අඩු කරන්න** (row සාමාන්‍යය, blank නොවන අගයන් විතරක් පාවිච්චි කරලා).

අඩු ratings → **ඍණ** සංඛ්‍යා; ඉහළ ratings → **ධන** සංඛ්‍යා.

ඊට පස්සේ cosine distance ගත්තොත්, දෙදෙනාම බලපු movies ගැන **ප්‍රතිවිරුද්ධ අදහස්** තියෙන users ලගේ vectors **බොහෝ දුරට ප්‍රතිවිරුද්ධ දිශාවලට** යනවා → පුළුවන් තරම් ඈත. **සමාන අදහස්** තියෙන users අතර **කුඩා කෝණයක්** → ඉහළ similarity.

Normalize කිරීමෙන් algorithm එකට users **වඩා හොඳින් වෙන් කර හඳුනාගන්න** උදව් වෙනවා.

පියවරෙන් පියවර (Figure 9.4 → Figure 9.6)

User	Ratings	සාමාන්‍යය	Normalized අගයන් (rating − සාමාන්‍යය)
A	4, 5, 1	(4+5+1)/3 = 10/3	HP1: 4 − 10/3 = (12−10)/3 = **2/3** TW: 5 − 10/3 = (15−10)/3 = **5/3** SW1: 1 − 10/3 = (3−10)/3 = **−7/3**
B	5, 5, 4	14/3	HP1: **1/3**, HP2: **1/3**, HP3: 4 − 14/3 = **−2/3**
C	2, 4, 5	11/3	TW: **−5/3**, SW1: **1/3**, SW2: **4/3**
D	3, 3	3	HP2: **0**, SW3: **0**

After normalizing: green = liked (above own average), red = disliked

	HP1	HP2	HP3	TW	SW1	SW2	SW3	
A	**2/3**			**5/3**	**-7/3**			avg 10/3
B	**1/3**	**1/3**	**-2/3**					avg 14/3
C				**-5/3**	**1/3**	**4/3**		avg 11/3
D		**0**					**0**	avg 3

Figure 9.6: සියලු ratings normalize කළ utility matrix එක (කොළ = තමන්ගේ සාමාන්‍යයට වඩා කැමතියි, රතු = අකමැතියි)

සිත්ගන්නා බලපෑමක්: D ගේ ratings ඇත්තටම **අතුරුදහන් වෙලා**, මොකද cosine distance ගණනය කරනකොට 0 කියන්නේ blank එකක් හා සමානයි. D දුන්නේ 3 විතරයි, movies අතර වෙනසක් කළේ නැහැ, ඒ නිසා D ගේ අදහස් බැරෑරුම් ලෙස සැලකිය යුතු නැති වෙන්න පුළුවන්.

Example 9.9 – normalize කළාට පස්සේ cosine

$$\cos(A, B) = (2/3 \times 1/3) \,/\, \left(\sqrt{(2/3)^2 + (5/3)^2 + (-7/3)^2} \times \sqrt{(1/3)^2 + (1/3)^2 + (-2/3)^2} \right) = 0.092$$

$$\cos(A, C) = (5/3 \times -5/3 + (-7/3) \times 1/3) \,/\, \left(\sqrt{(2/3)^2 + (5/3)^2 + (-7/3)^2} \times \sqrt{(-5/3)^2 + (1/3)^2 + (4/3)^2} \right) = -0.559$$

මේ මිනුමට අනුව **A සහ C, A සහ B ට වඩා ගොඩක් ඈතයි** (C ට සෘණ cosine එකක් පවා තියෙනවා, එනම් 124° පමණ කෝණයක්), සහ **කිසිම යුගලයක් ගොඩක් ළඟ නැහැ**. මේ නිරීක්ෂණ දෙකම බුද්ධියට එකඟයි: A සහ C පොදුවේ rate කරපු movies දෙක ගැන එකඟ නැහැ, A සහ B ඔවුන්ගේ පොදු movie එකට සමාන ලකුණු දුන්නා.

> **විභාග ඉඟිය (Exam tip)**
> මේවා අත්‍යවශ්‍ය පියවර. ඇත්ත recommender systems වලදී මේවා අතින් කරන්නේ නැහැ, නමුත් **න්‍යාය දැනගන්න ඕනේ** – මේ වගේ ගැටලු එකක් දෙකක් **අතින්** කරන්න අහන්න පුළුවන්, lab විභාගයේදී R වලින් කරන්න වෙයි.

2.6 පන්තියේ අභ්‍යාසය: R වලින් Figure 9.4 normalize කිරීම

දේශකතුමා ඇහුවා: **"Figure 9.4 හි matrix එක R පාවිච්චි කරලා normalize කරන්න"** (පැමිණීම සටහන් කරන්න R code එක එවන්නත් කිව්වා). කරන්න පුළුවන් එක් ක්‍රමයක්:

R code

```
# Figure 9.4 as a matrix (blank = NA)
M <- matrix(c(4, NA, NA, 5, 1, NA, NA,
              5, 5, 4, NA, NA, NA, NA,
              NA, NA, NA, 2, 4, 5, NA,
              NA, 3, NA, NA, NA, NA, 3), nrow = 4, byrow = TRUE)
rownames(M) <- LETTERS[1:4]
colnames(M) <- c("HP1","HP2","HP3","TW","SW1","SW2","SW3")

userMeans <- rowMeans(M, na.rm = TRUE)   # average of each user (ignore blanks)
userMeans
normM <- M - userMeans                   # subtract each user's average from their row
round(normM, 3)
```

Output (ප්‍රතිදානය)

```
       A        B        C        D
3.333333 4.666667 3.666667 3.000000
    HP1   HP2    HP3     TW    SW1   SW2 SW3
A 0.667    NA     NA  1.667 -2.333    NA  NA
B 0.333 0.333 -0.667     NA     NA    NA  NA
C    NA    NA     NA -1.667  0.333 1.333  NA
D    NA 0.000     NA     NA     NA    NA   0
```

0.667 = 2/3, 1.667 = 5/3, −2.333 = −7/3 … හරියටම Figure 9.6. (`M - userMeans` වැඩ කරන්නේ R, සාමාන්‍යයන් 4 තියෙන vector එක columns දිගේ නැවත නැවත පාවිච්චි කරන (recycle කරන) නිසා – row i ට mean i අඩු වෙනවා.)

2.7 Homework: normalize කළාට පස්සේ A සහ B අතර cosine distance

HW (whiteboard එකෙන්)

Normalize කළාට පස්සේ **A සහ B අතර cosine distance** සොයන්න: (I) **අතින්**, (II) **R පාවිච්චි කරලා**. දේශකතුමා කිව්වා: “මේවා අනිවාර්යයෙන්ම විභාග ප්‍රශ්න.”

(I) අතින්: Numerator: පොදු movie එක HP1 විතරයි → (2/3)(1/3) = 2/9 = 0.222. ‖A‖ = √(4/9 + 25/9 + 49/9) = √(78/9) = 2.944. ‖B‖ = √(1/9 + 1/9 + 4/9) = √(6/9) = 0.816. cos = 0.222 / (2.944 × 0.816) = **0.092**. Cosine distance = 1 − 0.092 = **0.908** (හෝ කෝණයක් විදිහට, arccos 0.092 ≈ 84.7°).

(II) R පාවිච්චි කරලා (ඉහත code එකෙන් ඉදිරියට):

R code

```
normZero <- normM
normZero[is.na(normZero)] <- 0          # blanks count as 0 for cosine
cosSim <- function(x, y) sum(x * y) / (sqrt(sum(x^2)) * sqrt(sum(y^2)))

cosSim(normZero["A", ], normZero["B", ])             # cosine similarity
1 - cosSim(normZero["A", ], normZero["B", ])         # cosine distance
acos(cosSim(normZero["A", ], normZero["B", ])) * 180 / pi   # angle in degrees
```

Output (ප්‍රතිදානය)

```
[1] 0.09245003
[1] 0.90755
[1] 84.69543
```

2.8 Homework 2 (Day 1 whiteboard): සියලු යුගල සඳහා Jaccard සහ cosine

Figure 9.4 පාවිච්චි කරලා (මුල් ratings, cosine සඳහා blanks = 0). ඔයාගේ උත්තර පරීක්ෂා කරන්න:

යුගලය	පොදු movies	Jaccard similarity	Jaccard distance	Cosine similarity
A, B	HP1	1/5 = 0.2	0.8	0.380
A, C	TW, SW1	2/4 = 0.5	0.5	0.322
A, D	නැහැ	0	1	0
B, C	නැහැ	0	1	0
B, D	HP2	1/4 = 0.25	0.75	15/(√66·√18) = 0.435
C, D	නැහැ	0	1	0

3. Exercise 9.3.1 සහ R පන්ති සැසිය

Figure 9.8 කියන්නේ utility matrix එකක්: users තුන් දෙනෙක් (A, B, C) අයිතම අටක් (a සිට h) 1–5 තරු පරිමාණයෙන් rate කරලා. මේක **විභාගයට ඉතා ගැලපෙන අභ්‍යාසයක්** – අතින් කරන්නත්, R වලින් function එකක් ලියන්නත් අහන්න පුළුවන්.

	a	b	c	d	e	f	g	h
A	4	5		5	1		3	2
B		3	4	3	1	2	1	
C	2		1	3		4	5	3

මේ matrix එකෙන් පහත දේවල් ගණනය කරන්න:

a. Utility matrix එක **boolean** විදිහට (rate කළා / කළේ නැහැ) සලකලා, එක් එක් users යුගලය අතර **Jaccard distance** ගණනය කරන්න.

b. Part (a) නැවත කරන්න, නමුත් **cosine distance** පාවිච්චි කරන්න.

c. Ratings **3, 4, 5 → 1** සහ **1, 2 සහ blank → 0** ලෙස සලකන්න. එක් එක් users යුගලය අතර **Jaccard distance** ගණනය කරන්න.

d. Part (c) නැවත කරන්න, නමුත් **cosine distance** පාවිච්චි කරන්න.

e. Blank නොවන හැම අගයකින්ම ඒ user ගේ සාමාන්‍යය අඩු කරලා matrix එක **normalize** කරන්න.

f. Part (e) හි normalized matrix එක පාවිච්චි කරලා, එක් එක් users යුගලය අතර **cosine distance** ගණනය කරන්න.

> **Statistics සමඟ සම්බන්ධය (දේශකතුමාගේ පැති අදහස)**
> මෙතැන users තුන් දෙනෙක් එකම අයිතම rate කරනවා – statistics වල සම්බන්ධිත (paired) data වගේ. මතක් කරගන්න: ස්වාධීන treatments 2ක් සඳහා **t-test**; 2 ට වැඩි නම් **ANOVA** (mean comparison). සම්බන්ධිත samples සඳහා paired versions පාවිච්චි කරනවා, **parametric assumptions සපුරන්නේ නැති නම්** distributions සංසන්දනය කරන **non-parametric tests** වලට යනවා. ඔයා statistics සමස්තයක් විදිහට ඉගෙනගන්න නිසා මේ පැතිත් ගැන හිතන්න.

3.1 පන්තියේදී: R වලින් part (c) – පියවරෙන් පියවර

දේශකතුමා ශිෂ්‍යයන්ට **part (c)** R වලින් කරලා code එක එවන්න කිව්වා (සහභාගී වෙන්න නම් කිහිපයක් කතා කළා). මේ machine-learning පන්තියට R වලට කැමතියි කියලා ශිෂ්‍යයන් කිව්වා, ඒ නිසා **මේ වගේ පොඩි දේවල් R වලින් කරන්න පුළුවන් වෙන්න ඕනේ**.

පියවර 1 – data ටික matrix එකක ගබඩා කිරීම

R code

```
ratings = matrix(c(
  4, 5, NA, 5, 1, NA, 3, 2,
  NA, 3, 4, 3, 1, 2, 1, NA,
  2, NA, 1, 3, NA, 4, 5, 3
), nrow = 3, byrow = TRUE)
ratings
```

- `matrix()` තමයි command එක; මුලින්ම data දෙනවා.
- **Blanks** සාමාන්‍යයෙන් `NA` (missing values) විදිහට සලකනවා.

- `nrow = 3` – rows ගණන. `ncol` දීම අනිවාර්ය නැහැ – data සහ rows ගණන දුන්නාම R ට columns ගණන තේරුම් ගන්න පුළුවන්.
- `byrow = TRUE` කියන්නේ matrix එක **row අනුව පුරවනවා**: R පළමු row එක පුරවලා, දෙවන row එකට යනවා, එහෙම ඉදිරියට.

> **සටහන**
>
> **පරිස්සමෙන්:** පන්තියේ code එකේ B සහ C rows type කරලා තිබුණේ table එකට වඩා වෙනස් විදිහට (B = 3, 4, 3, 1, 2, 1, NA, NA සහ C = 2, 1, 3, NA, 4, 5, 3, NA – blanks වැරදි තැන්වල). Figure 9.8 ට ගැලපෙන නිවැරදි rows ඉහත පෙන්වලා තියෙනවා: B = NA, 3, 4, 3, 1, 2, 1, NA සහ C = 2, NA, 1, 3, NA, 4, 5, 3. හැම වෙලේම blanks තියෙන තැන් පරීක්ෂා කරන්න!

පියවර 2 – column names සහ row names එකතු කිරීම (අනිවාර්ය නැහැ)

R code

```
colnames(ratings)            # no column names yet -> NULL
letters                      # "a" "b" "c" ... "z"  (built-in, lower case)
LETTERS                      # "A" "B" "C" ... "Z"  (built-in, capital)
colnames(ratings) = c(letters[1:8])    # a, b, ..., h
rownames(ratings) = LETTERS[1:3]       # A, B, C
ratings
```

Output (ප්‍රතිදානය)

```
   a  b  c d  e  f g  h
A  4  5 NA 5  1 NA 3  2
B NA  3  4 3  1  2 1 NA
C  2 NA  1 3 NA  4 5  3
```

නම් කියන්නේ අමතර row එකක් නෙවෙයි – තවමත් rows 3 යි; ඒවා labels විතරයි. මේකෙන් matrix එක හරියටම ප්‍රශ්නයේ වගේ පේනවා. **මේක අනිවාර්ය නැහැ – නම් එකතු නොකළත් ලකුණු අඩු කරන්නේ නැහැ කියලා දේශකතුමා කිව්වා.**

පියවර 3 – element-wise සංසන්දනය සහ ifelse()

R ගණනය කිරීම් කරන්නේ **element-wise** (එක් එක් අගයට වෙන වෙනම). `ratings > 1` ලිව්වොත් R **හැම අගයක්ම** 1 සමඟ සංසන්දනය කරලා TRUE හෝ FALSE දෙනවා (boolean matrix එකක්). 4 > 1 TRUE, 1 > 1 FALSE, NA, NA ම වෙනවා:

R code

```
ratings > 1
```

Output (ප්‍රතිදානය)

```
     a    b     c    d     e    f     g    h
A TRUE TRUE    NA TRUE FALSE   NA  TRUE TRUE
B   NA TRUE  TRUE TRUE FALSE TRUE FALSE   NA
C TRUE   NA FALSE TRUE    NA TRUE  TRUE TRUE
```

ඒ නිසා `ifelse()` පාවිච්චි කරනවා: rating එක 3 හෝ ඊට වැඩි නම් 1, නැත්නම් 0. (නම් තැබීම ගැන සටහනක්: අය විවිධ ක්‍රම පාවිච්චි කරනවා, උදා: `binary_ratings` හෝ `binaryRatings` – දෙවැන්න **camel notation** (දෙවන වචනය කැපිටල් අකුරෙන්), දේශකතුමා සාමාන්‍යයෙන් පාවිච්චි කරන්නේ ඒක. ඕනෑම එක් ක්‍රමයකට එකඟව ඉන්න.)

R code

```
binaryRatings = ifelse(ratings >= 3, 1, 0)
binaryRatings
```

Output (ප්‍රතිදානය)

```
   a  b  c d  e  f g  h
A  1  1 NA 1  0 NA 1  0
B NA  1  1 1  0  0 0 NA
C  0 NA  0 1 NA  1 1  1
```

පියවර 4 – missing values (NA) වෙනුවට 0 දැමීම

ප්‍රශ්නය කියනවා blanks ත් 0 වෙන්න ඕනේ කියලා, නමුත් තවමත් NA තියෙනවා. `is.na(binaryRatings)` TRUE/FALSE matrix එකක් දෙනවා (අගය missing තැන්වල TRUE). පළමු උත්සාහය `is.na(binaryRatings) = 0` වැඩ කළේ නැහැ – `is.na(...)` **square brackets ඇතුළට** දාන්න ඕනේ, එතකොට R හරියටම ඒ තැන්වලට ගිහින් 0 දානවා (ශිෂ්‍යයෙක් මේ උත්තරය හොයාගත්තා):

R code

```
binaryRatings[is.na(binaryRatings)] = 0
binaryRatings
```

Output (ප්‍රතිදානය)

```
  a b c d e f g h
A 1 1 0 1 0 0 1 0
B 0 1 1 1 0 0 0 0
C 0 0 0 1 0 1 1 1
```

දැන් binary matrix එක තියෙනවා. → A කැමති {a, b, d, g}; B කැමති {b, c, d}; C කැමති {d, f, g, h}.

පියවර 5 – එක් එක් යුගලය සඳහා Jaccard distance

Jaccard distance = 1 − (intersection / union). ඒක **හැම යුගලයකටම** ඕනේ. යුගල කීයක්ද? Users 3 දෙනෙක් එක්ක: **AB, AC සහ BC** → distances 3ක්. ඒ නිසා utility matrix එක සහ users දෙදෙනා (a සහ b) ගන්න **function** එකක් ලියන එක හොඳයි:

R code

```
# attempt shown in class (does NOT work correctly yet)
JaccardDist = function(UM, A, B){
  intersect(UM[A,], UM[B,]) / union(UM[A,], UM[B,])
}
intersect(binaryRatings[1,], binaryRatings[2,])
union(binaryRatings[1,], binaryRatings[2,])
```

Output (ප්‍රතිදානය)

```
[1] 1 0
[1] 1 0
```

ඒක වැඩ නොකරන්නේ ඇයි: `intersect()` සහ `union()` වැඩ කරන්නේ vectors දෙකේ තියෙන **වෙනස් අගයන්ගේ කට්ටලය** මත. Rows දෙකේම තියෙන්නේ 0 සහ 1 අගයන් විතරයි, ඒ නිසා ප්‍රතිඵලය “1 0” විතරයි – ඒක කිසිවක් ගණන් කරන්නේ නැහැ. (දේශකතුමා `intersect()` උදාහරණ හොයාගත්තේ ChatGPT අහලා නෙවෙයි, R help/example code එකෙන්.)

ඇත්තටම ඕනේ **column එකින් column එක** ගිහින්, දෙකම 1 තියෙන **තැන් ගණන් කරන්න** (intersection) සහ අවම වශයෙන් එකක් 1 තියෙන තැන් ගණන් කරන්න (union). දේශකතුමාගේ උදාහරණයේ (data වල වෙනත් version එකක) පොදු තැන් 5ක් 8න් තිබුණා, 5/8 ලැබෙනවා – කාරණය: **වෙනස් අගයන් නෙවෙයි, සිදුවීම් ගණන (number of occurrences) ගන්න**.

HOMEWORK (සතියකින් භාර දෙන්න, LMS එකේ link එකක් open කරනවා)

Part (c) සඳහා එක් එක් users යුගලය (AB, AC, BC) අතර Jaccard distance ගණනය කරන්න **ඔයාගේම** R function එකක් ලියන්න. මුලින්ම තනියම උත්සාහ කරන්න – විභාගයේදී ChatGPT හෝ කිසිම AI tool එකක් පාවිච්චි කරන්න බැහැ.

එක් හැකි විසඳුමක් (උත්සාහ කළාට පස්සේ ඔයාගේ එක පරීක්ෂා කරන්න):

R code

```
JaccardDist = function(UM, A, B){
  inter <- sum(UM[A, ] == 1 & UM[B, ] == 1)   # both liked
  uni   <- sum(UM[A, ] == 1 | UM[B, ] == 1)   # at least one liked
  1 - inter / uni
}
JaccardDist(binaryRatings, "A", "B")
JaccardDist(binaryRatings, "A", "C")
JaccardDist(binaryRatings, "B", "C")
```

Output (ප්‍රතිදානය)

```
[1] 0.6
[1] 0.6666667
[1] 0.8333333
```

සටහන

AI tools ගැන: එවපු සමහර උත්තර AI tool එකකින් ගත්ත ඒවා බව දේශකතුමා දැක්කා (ශිෂ්‍යයන් දෙදෙනෙක් හරියටම එකම function එක දුන්නා; එක උත්තරයක missing values handle කරලාවත් තිබුණේ නැහැ, මොකද ChatGPT එකට upload කරපු screenshot එකේ blanks පෙන්නුවේ නැහැ). මේ වෙලාවේ ලකුණු අඩු කළේ නැහැ, නමුත් අවංක වෙන්න සහ තමන්ගේම code ලියන්න කිව්වා – නැත්නම් code කටපාඩම් කරනවා විතරයි, විභාගයේදී අලුත් ප්‍රශ්න විසඳන්න බැරි වෙනවා.

3.2 Exercise 9.3.1 හි සියලු කොටස් සඳහා උත්තර (R වලින් පරීක්ෂා කළා)

මෙතැන cosine distance = 1 − cosine similarity (cosine similarity වරහන් තුළ).

යුගලය	(a) Jaccard, boolean	(b) Cosine, මුල් ratings	(c) Jaccard, 3–5 → 1	(d) Cosine, 3–5 → 1	(f) Cosine, normalized
A, B	4/8 → 0.5	0.399 (0.601)	2/5 → 0.6	0.423 (0.577)	0.416 (0.584)
A, C	4/8 → 0.5	0.385 (0.615)	2/6 → 0.667	0.500 (0.500)	1.115 (−0.115)
B, C	4/8 → 0.5	0.486 (0.514)	1/6 → 0.833	0.711 (0.289)	1.740 (−0.740)

(e) Normalized matrix එක (users ලගේ සාමාන්‍යයන්: A = 10/3, B = 7/3, C = 3):

	a	b	c	d	e	f	g	h
A	2/3	5/3		5/3	−7/3		−1/3	−4/3
B		2/3	5/3	2/3	−4/3	−1/3	−4/3	
C	−1		−2	0		1	2	0

(a) සඳහා කට්ටල කියන්නේ rate කරපු අයිතම: A = {a,b,d,e,g,h}, B = {b,c,d,e,f,g}, C = {a,c,d,f,g,h}; හැම යුගලයක්ම union එක 8 න් අයිතම 4ක් බෙදාගන්නවා.

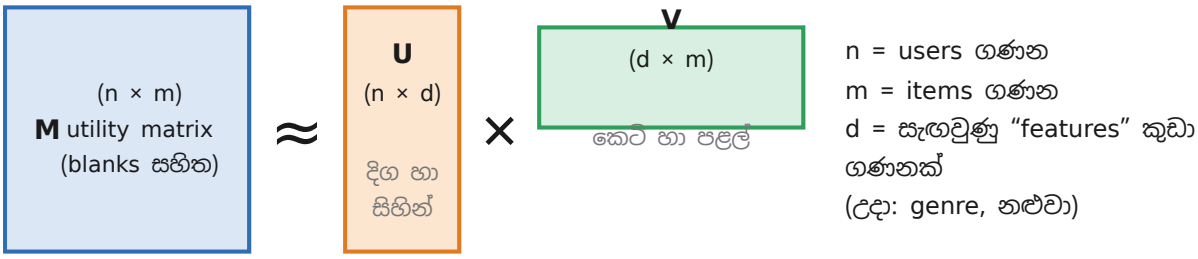
4. Dimensionality reduction: UV-decomposition

අපි කලින් **unsupervised learning** වලදී **dimension reduction** ගැන කතා කළා (graphical ක්‍රම, සහ dimensions අඩු කරනකොට image quality එක අඩු වෙන හැටි – පූසාගේ පින්තූරය). මෙතැන අපි පාවිච්චි කරන්නේ **UV-decomposition**, ඒක **SVD (singular-value decomposition)** කියන පුළුල් න්‍යායේ විශේෂ අවස්ථාවක්.

අදහස: utility matrix එකේ **බොහෝ** අගයන් හිස්. අපි utility matrix එක, වැඩ කරන්න පහසු matrices දෙකක් U සහ V බවට **වියෝජනය (decompose)** කරනවා – blanks එක්ක වැඩ කරන්න එක් ක්‍රමයක්.

4.1 අදහස

- Movies උදාහරණයට ගමු. **බොහෝ users ප්‍රතිචාර දක්වන්නේ features කුඩා ගණනකට**: ඔවුන් සමහර **genres** වලට කැමතියි, සමහර **ප්‍රසිද්ධ නළු නිළියන්ට** කැමති වෙන්න පුළුවන්, සමහර විට සැලකිය යුතු රසික පිරිසක් ඉන්න **අධ්‍යක්ෂවරුන්** කිහිප දෙනෙක් ඉන්නවා.
- **n rows** (users) සහ **m columns** (items) තියෙන utility matrix එක **M** ගෙන් පටන් ගන්න.
- M හි **blank නොවන** අගයන්වලදී **UV, M ට ආසන්න වශයෙන් සමාන** වෙන විදිහට, **n rows සහ d columns තියෙන U** matrix එකක් සහ **d rows සහ m columns තියෙන V** matrix එකක් හොයාගන්න පුළුවන් වෙන්න පුළුවන්.
- එහෙනම් users සහ items දෙකම හොඳින් විස්තර කරන **d dimensions** (සැඟවුණු features) තියෙනවා. ඊට පස්සේ M හි **blank අගයන් ඇස්තමේන්තු කරන්න** UV ගුණිතයේ අගයන් පාවිච්චි කරනවා.
- මේ ක්‍රියාවලියට M හි **UV-decomposition** කියනවා.



ගණිතයෙන් මතක් කරගන්න: matrices දෙකක් ගුණ කරන්න නම් **පළමු එකේ columns ගණන දෙවැන්නේ rows ගණනට** සමාන වෙන්න ඕනේ. (n × d) × (d × m) = (n × m) matrix එකක් – M ගේ ප්‍රමාණයමයි.

4.2 Example 9.11 – 5 × 5 matrix එකක්

M කියන්නේ අගයන් **දෙකක් හැර** අනෙක් සියල්ල දන්නා 5 × 5 matrix එකක්. අපිට ඒක **5 × 2** matrix එකක් U සහ **2 × 5** matrix එකක් V බවට වියෝජනය කරන්න ඕනේ (ඉතින් d = 2 තෝරගත්තා). U සහ V හි අගයන් (u_{11}, u_{12}, ..., v_{11}, ...) සොයාගත යුතු විචල්‍යයන්.

5	2	4	4	3
3	1	2	4	1
2	?	3	1	4
2	5	4	3	5
4	4	5	4	?

=

u11	u12
u21	u22
u31	u32
u41	u42
u51	u52

×

v11	v12	v13	v14	v15
v21	v22	v23	v24	v25

Figure 9.9: M matrix එකේ UV-decomposition ("?" කොටු දෙක blanks).

මේක **U සහ V හි මුළු අගයන් ගණනට (20) වඩා දන්නා අගයන් (23) වැඩි** සරලම අවස්ථාව, ඒ නිසා හොඳම වියෝජනයෙන් M හි blank නොවන අගයන් හරියටම ලැබෙන්නේ **නැහැ** – M ට

ආසන්න අගයක් විතරයි ලැබෙන්නේ.

4.3 Root-mean-square error (RMSE)

UV, M ට කොච්චර ළඟද කියලා මනින්න මිනුමක් ඕනේ. සාමාන්‍ය තේරීම **RMSE**:

1. M හි **blank නොවන** සියලු අගයන් සඳහා, ඒ අගය සහ UV හි අනුරූප අගය අතර **වෙනසේ වර්ගය එකතු කරන්න**.
2. එකතු කළ පද ගණනින් (M හි blank නොවන අගයන් ගණන) බෙදලා මේ වර්ගවල **සාමාන්‍යය (mean)** ගන්න.
3. සාමාන්‍යයේ **වර්ගමූලය** ගන්න.

වර්ගවල එකතුව අවම කිරීම RMSE අවම කිරීමට සමානයි, ඒ නිසා බොහෝ විට පියවර 1 න් නවතිනවා. **RMSE අඩු වෙන තරමට හොඳයි.**

Example 9.12 – පළමු (දුර්වල) අනුමානය: සියල්ල 1

U සහ V හි හැම අගයක්ම 1 කියලා අනුමාන කරමු. UV හි හැම අගයක්ම = 1×1 + 1×1 = **2**, ඉතින් UV කියන්නේ සියල්ල 2 තියෙන 5 × 5 matrix එකක්. මේක **දුර්වල අනුමානයක්** – M හි අගයන් (5, 4, 3 ...) බොහෝ විට 2 ට වඩා ගොඩක් විශාලයි – නමුත් RMSE තේරුම් ගන්න පහසුයි.

M හි row එක	M − UV (වෙනස්කම්, blanks නොසලකා)	වර්ග	වර්ගවල එකතුව
1: 5 2 4 4 3	3, 0, 2, 2, 1	9, 0, 4, 4, 1	18
2: 3 1 2 4 1	1, −1, 0, 2, −1	1, 1, 0, 4, 1	7
3: 2 _ 3 1 4	0, (blank), 1, −1, 2	0, 1, 1, 4	6
4: 2 5 4 3 5	0, 3, 2, 1, 3	0, 9, 4, 1, 9	23
5: 4 4 5 4 _	2, 2, 3, 2, (blank)	4, 4, 9, 4	21
එකතුව			**75**

Row 3 හි දෙවන column එක blank, row 5 හි අන්තිම column එක blank – RMSE ගණනය කරනකොට ඒ අගයන් **නොසලකා හරිනවා**.

වර්ග කළ දෝෂවල එකතුව = 18 + 7 + 6 + 23 + 21 = **75**. Blank නොවන අගයන් ගණන = 25 − 2 = **23**. ඉතින් RMSE = √(75/23) = √3.26 = **1.806**.

R code (RMSE පරීක්ෂා කිරීම)

```
M <- matrix(c(5,2,4,4,3, 3,1,2,4,1, 2,NA,3,1,4, 2,5,4,3,5, 4,4,5,4,NA), 5, byrow = TRUE)
UV <- matrix(1, 5, 2) %*% matrix(1, 2, 5)     # all entries = 2
rowSums((M - UV)^2, na.rm = TRUE)             # 18  7  6 23 21
sqrt(mean((M - UV)^2, na.rm = TRUE))          # RMSE = 1.805788
```

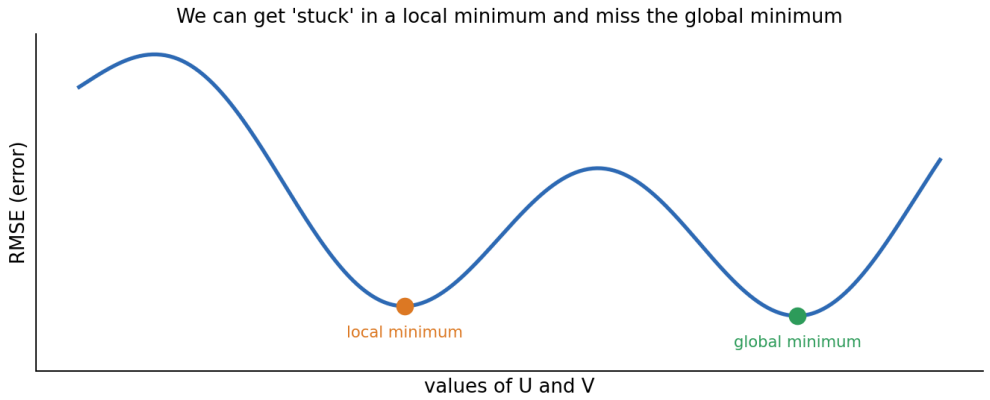
RMSE එකෙන් බලන්නේ UV, ඇත්ත M ගෙන් කොච්චර **ඈතද** කියලා. අපිට U සහ V **පුනරාවර්තනව (iteratively)** වෙනස් කරලා, හැම වතාවකම RMSE ගණනය කරලා, හොඳම එක තෝරගන්න පුළුවන්.

4.4 U සහ V සොයාගැනීම (දැනුමට – විභාගයට එන්නේ නැහැ)

විභාග ඉඟිය (Exam tip)

මේ RMSE optimisation කොටස ටිකක් සංකීර්ණයි, ඒක **විභාගයට එන්නේ නැහැ** කියලා දේශකතුමා කිව්වා. කලින් කොටස් (Jaccard, cosine, rounding, normalizing) **විභාගයට එනවා**.

- **අහඹු ලෙස තෝරගත්** U සහ V (උදා: සියල්ල 1) වලින් පටන් අරන්, RMSE අඩු වෙන විදිහට ඒවා **නැවත නැවත සකස් කරනවා**.
- අපි **වරකට එක් අගයක්** වෙනස් කරනවා: එක් අගයක් x විචල්‍යයක් කරලා, වර්ග කළ දෝෂවල එකතුව x හි function එකක් ලෙස ලියලා, අවකලනය (differentiate) කරලා, 0 ට සමාන කරලා විසඳනවා. Example 9.13: u_{11} = x කළාම පළමු row එකේ දෝෂය $(4 - x)^2 + (1 - x)^2 + (3 - x)^2 + (3 - x)^2 + (2 - x)^2 \rightarrow x = \mathbf{2.6}$, එයින් පළමු row එකේ දෝෂය 18 සිට 5.2 දක්වා, මුළු එකතුව 75 සිට 62.2 දක්වා අඩු වෙනවා. ඊට පස්සේ $v_{11} = y \rightarrow y = 1.617$, $u_{31} = z \rightarrow z = 1.178$, එහෙම ඉදිරියට.
- **Local minima ගොඩක්** තියෙනවා – කිසිම තනි වෙනසකින් RMSE තවත් අඩු නොවන U සහ V අගයන්. ඒවායින් **එකක්** විතරයි **global minimum** එක. අපි නොදැනම local minimum එකක හිරවෙන්න පුළුවන් – optimization වල සාමාන්‍ය ප්‍රශ්නයක්. අවස්ථාව වැඩි කරගන්න **විවිධ ආරම්භක ලක්ෂ්‍ය ගොඩකින්** පටන් ගන්න.



Local minimum එකක හිරවෙලා global minimum එක මඟහැරෙන්න පුළුවන් (x අක්ෂය: U සහ V අගයන්, y අක්ෂය: RMSE දෝෂය)

සම්පූර්ණ UV-decomposition algorithm එකක් හැදීමේදී තීරණ හතරක් තියෙනවා:

- **පූර්ව සැකසීම (Preprocessing)** – මුලින්ම M normalize කරන්න (users ලගේ සහ/හෝ items වල සාමාන්‍යයන් අඩු කරන්න); predict කරනකොට normalization එක **ආපසු හරවන්න (undo)** මතක තියාගන්න.
- **ආරම්භ කිරීම (Initialisation)** – උදා: U සහ V හි හැම අගයකටම $\sqrt{(a/d)}$ දෙන්න; a = M හි blank නොවන අගයන්ගේ සාමාන්‍යය. විවිධ ආරම්භක ලක්ෂ්‍ය සඳහා අහඹු noise එකතු කරන්න.
- **Optimise කරන පිළිවෙළ** – අගයන් row අනුව round-robin විදිහට, නැත්නම් අහඹු පිළිවෙළකට; නැවත නැවත බලන්න.
- **නැවැත්වීම** – එක් වටයකදී RMSE හි දියුණුව සීමාවකට (threshold) වඩා අඩු වුණාම නවත්වන්න.

මේ ක්‍රමය **gradient descent** සඳහා උදාහරණයක් (හැම වතාවකම data වලින් අහඹු කොටසක් විතරක් බලන එකට **stochastic gradient descent** කියනවා). **Overfitting** වළක්වන්න (දී ඇති data වල RMSE කුඩායි, නමුත් අනාගත data හොඳින් predict කරන්නේ නැහැ): එක් එක් අගය එහි optimum එක දක්වා කොටසක් විතරක් ගෙනයන්න, සම්පූර්ණයෙන් converge වෙන්න කලින් නවත්වන්න, නැත්නම් විවිධ UV-decompositions කිහිපයක ප්‍රතිඵලවල සාමාන්‍යය ගන්න. දේශන සටහන් (PDF) වල මේ සඳහා R routines ත් තියෙනවා – ඒවා කියවන්න.

5. Netflix Challenge

Netflix Challenge පටන් ගත්ත හැටි පසුගිය වතාවේ කතා කළා. Recommender systems පර්යේෂණවලට ලොකු තල්ලුවක් ලැබුණේ Netflix තමන්ගේම recommendation algorithm එක වන **CineMatch** එක **10%** කින් පරාජය කරන පළමු පුද්ගලයාට හෝ කණ්ඩායමට **ඩොලර් 1,000,000 ත්‍යාගයක්** පිරිනැමුවාම. වසර තුනකට වැඩි වැඩකින් පස්සේ ත්‍යාගය **2009 සැප්තැම්බර්** මාසයේ ප්‍රදානය කළා.

- Data: users **මිලියන භාගයක්** පමණ, movies **17,000** පමණ (ඉන් කුඩා කොටස්) වලට දුන් ratings, එක් එක් එකට rating එක (තරු 1–5) සහ දිනය. Algorithms පරීක්ෂා කළේ data වල **රහසිගත** කොටසක් මත.
- **Algorithms වල කාර්ය සාධනය මනින්න RMSE පාවිච්චි කළා** – අපි RMSE ඉගෙනගත්තේ ඒ නිසා: recommender systems **සංසන්දනය කරන්න** හොඳ metric එකක්.
- CineMatch හි RMSE එක **0.95** පමණ – සාමාන්‍ය prediction එකක් තරුවක් පමණ වැරදුණා. දිනන්න නම් ඔයාගේ RMSE එක CineMatch ගේ RMSE එකෙන් **90% ට වඩා අඩු** වෙන්න ඕනේ.
- **CineMatch එතරම් හොඳ algorithm එකක් නෙවෙයි.** ඉතා සරල algorithm එකක් – user u, movie m ට දෙන rating එක predict කරන්නේ (1) u rate කරපු සියලු movies වලට u දුන් සාමාන්‍ය rating එක (**පුද්ගලයා අනුව සාමාන්‍යය**) සහ (2) m rate කරපු සියලු users m ට දුන් සාමාන්‍ය rating එක (**movie එක අනුව සාමාන්‍යය**) යන දෙකේ **සාමාන්‍යය** ලෙස – CineMatch ට වඩා **3% ක් විතරයි නරක**.
- අපි දැන් ඉගෙනගත්ත **UV-decomposition** algorithm එක, normalization සහ තවත් උපක්‍රම කිහිපයක් එක්ක, CineMatch ට වඩා **7% දියුණුවක්** දෙන බව **ශිෂ්‍යයන් තුන් දෙනෙක්** (Michael Harris, Jeffrey Wang සහ David Kamm) සොයාගත්තා. (සල්ලි ගොඩක් හිතන්න පොළඹවනවා – ඩොලර් මිලියනයක් කියන්නේ විශේෂයෙන් අවුරුදු 15–20 කට කලින් ලොකු ධනයක්!)
- **දිනපු ඉදිරිපත් කිරීම** ස්වාධීනව හදපු **විවිධ algorithms කිහිපයක එකතුවක්**. **දෙවන කණ්ඩායමකුත්** algorithms එකතුවක් ඉදිරිපත් කළා, ඒක **විනාඩි කිහිපයකට කලින් ඉදිරිපත් කළා නම් දිනනවා**. Algorithms එකතු කිරීම මතක තියාගන්න වටින උපක්‍රමයක්.
- **IMDb** (Internet Movie Database) තොරතුරු (නළුවන්, අධ්‍යක්ෂවරුන්, genres 28) පාවිච්චි කරන්න කළ උත්සාහ **ප්‍රයෝජනවත් වුණේ නැහැ** – algorithms ඒ තොරතුරු කොහොමත් සොයාගත්තා වෙන්න පුළුවන්, Netflix සහ IMDb අතර movie නම් ගලපන එකත් අමාරුයි.
- **Rate කළ වේලාව** ප්‍රයෝජනවත් වුණා – සමහර movies (උදා: “Patch Adams”) බලපු ගමන් rate කරන අය වැඩිපුර කැමතියි, තවත් සමහර ඒවා (උදා: “Memento”) ටික කාලයකට පස්සේ වැඩිපුර අගය කරනවා.

> **Deadlines ගැන දේශකතුමාගේ පාඩම**
>
> “මම deadlines වලට තදින් ඇලිලා ඉන්නේ මේ නිසයි.” ඇත්ත ලෝකයේ deadline එකක් මඟහැරුණොත් ඒක මඟහැරුණාමයි. උදාහරණය: factory එකක shipment එකක් සැලසුම් කරනකොට container එක port එකට ගිහින් නැවට යන්න සවස 5:00 ට පිටත් වෙන්න ඕනේ; ඒක මඟහැරුණොත් **air freight** වලින් යවන්න වෙනවා – ගොඩක් වැඩි වියදමක් – ඔයාගේ රැකියාවත් නැති වෙන්න පුළුවන්. අර දෙවන Netflix කණ්ඩායමට විනාඩි කිහිපයකින් ඩොලර් මිලියනයක් අහිමි වුණා. **දේශකතුමාගේ deadlines ස්ථිරයි.**

ඉගෙනීම ගැන දේශකතුමාගේ උපදෙස් (සහ ඔයාගේ ජීවිතයේ recommender algorithms)

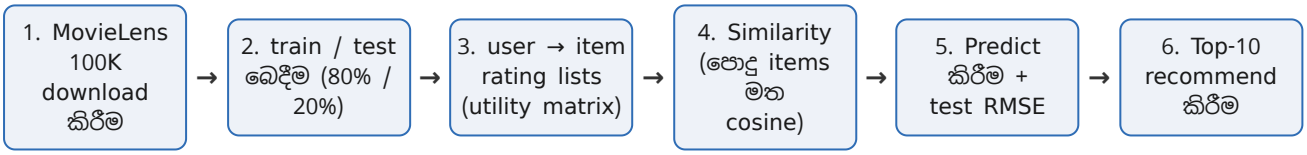
YouTube එකේ "Netflix challenge winners" කියලා search කරන්න – documentaries තියෙනවා. Internet එක දුර්වල හෝ මිල අධික නම්, lab එකේදී videos download කරලා phone එකට දාගෙන ගමන් කරනකොට (උදා: bus එකේ) බලන්න. ඉගෙනගන්නේ එහෙමයි.

දේශකතුමා උපාධිය ගත්තකොට (2008, විද්‍යා පීඨය) පුස්තකාලයේ අදාළ පොත් ඉතා ස්වල්පයයි, internet තියෙන computers එකක් දෙකක් විතරයි, දිගු පෝලිම්. අද phone එකෙන් වුණත් internet තියෙනවා – නමුත් අපි ලේසියෙන් **අවධානය වෙනතකට යොමු වෙනවා**.

සමාගම් අපිව නිරීක්ෂණය කරලා අපේ data සහ කැමැත්ත විකුණනවා. ඔවුන්ගේ recommender algorithms අපි බලන දේවලින් ඉගෙනගන්නවා: ඔයා cooking videos හෝ කාලය නාස්ති කරන short videos බැලුවොත්, algorithm එක ඒ වගේම දේවල් තවත් එවනවා, ඔයා හිරවෙනවා. අධ්‍යාපනික pages සහ job-posting sites follow කළොත් ඒවා එවනවා. **ඔයාගේ algorithm එක හරවගන්න (tweak කරන්න)** – කාලය නාස්ති කරන්න එපා; මේ ඔයාගේ අවසාන අවුරුද්ද වෙන්න පුළුවන්, උපාධිය ගන්න කලින් ඒ පරතරය පුරවගන්න.

6. R වලින් මුල සිටම recommender system එකක් හැදීම

දේශනය අවසානයේ **MovieLens 100K** data set එක (movie ratings) පාවිච්චි කරලා මුල සිටම recommender system එකක් හැදුවා. දේශකතුමා RStudio පාවිච්චි කරලා code එක පන්තියත් එක්ක බෙදාගත්තා. (ඔහු කිව්වා ඒ වෙලාවේ ලියනවා නෙවෙයි, කලින් හදපු code එකක් පාවිච්චි කරනවා කියලා – **මේ සියල්ල තනියම ලියන්න ඕනේ නැහැ**, නමුත් එක් එක් පියවර තේරුම් ගන්න ඕනේ.)



6.1 Data ලබාගැනීම

R code

```
# Create a workspace
dir.create("ml-100k", showWarnings = FALSE)

# Download MovieLens 100K
zip_url <- "https://files.grouplens.org/datasets/movielens/ml-100k.zip"
zip_file <- "ml-100k.zip"
download.file(zip_url, zip_file, mode = "wb")
unzip(zip_file, exdir = ".")

# Load ratings (u.data: user_id, item_id, rating, timestamp)
ratings <- read.table("ml-100k/u.data", sep = "\t",
                      col.names = c("user", "item", "rating", "ts"))
head(ratings)
```

- මුලින්ම `getwd()` එකෙන් working directory එක බලන්න; data එතැනට save වෙන්න දේශකතුමා ඒක source file එකේ folder එකට (ඔහුගේ "Rajarata D6" folder එකට) set කළා.
- `download.file()` zip file එක download කරනවා, `unzip()` ඒක extract කරනවා – දෙකම **R ඇතුළෙන්ම**. Data files කිහිපයක් සහිත `ml-100k` folder එකක් හැදෙනවා.
- Ratings තියෙන්නේ `u.data` file එකේ (tab-separated). Data description file එකේ data ගැන සියල්ල කියලා තියෙනවා – ඒක අනුගමනය කරන්න.
- හැම row එකකම: **user** ID, **item** (movie) ID, **rating** (1–5) සහ **ts** (timestamp).

6.2 Data train සහ test ලෙස බෙදීම

R code

```
set.seed(42)          # so the random split can be reproduced

split_by_user <- function(df, test_frac = 0.2) {
  idx_test <- unlist(tapply(seq_len(nrow(df)), df$user, function(ix) {
    n <- length(ix)
    n_test <- max(1, floor(n * test_frac))
    sample(ix, n_test)
  }))
  test <- df[idx_test, ]
  train <- df[-idx_test, ]
  list(train = train, test = test)
}

spl <- split_by_user(ratings, 0.2)
train <- spl$train
test  <- spl$test

dim(ratings)   # 100000 rows and 4 cols
dim(train)     # 80367     4
dim(test)      # 19633     4
```

- `set.seed(42)` අහඹු sampling එක නැවත එලෙසම ලබාගන්න (reproduce කරන්න) උදව් වෙනවා.
- **හැම user කෙනෙකුටම** (`tapply` row numbers user අනුව group කරනවා), ඒ user ගේ ratings වලින් 20% පමණ (අවම වශයෙන් 1) අහඹු ලෙස **test** set එකට තෝරනවා; ඉතිරිය **train** set එකට. එතකොට හැම user ම sets දෙකේම ඉන්නවා.
- සම්පූර්ණ data වල **rows 100,000 සහ columns 4** ක් තියෙනවා; train එකේ 80% පමණ (80,367), test එකේ ඉතිරි 19,633.

6.3 User-based collaborative filtering පියවර

1. **User–item matrix** එකක් (utility matrix) හදන්න.
2. **Users අතර similarity** ගණනය කරන්න (cosine similarity, Jaccard, හෝ Pearson correlation).
3. අසල්වැසියන්ගේ (neighbours) ratings වලින් **predict කරන්න**.

Data තියෙන්නේ “long” ආකාරයෙන් (එක් rating එකකට එක් row එකක්: user, item, rating). අපිට ඕනේ ආකාරය ඒක නෙවෙයි – utility matrix එකට **එක පැත්තක users, අනෙක් පැත්තේ items, සහ එක් එක් item එකට rating එක** ඕනේ. (පන්තියේදී මේක පැහැදිලිද කියලා ඇහුවා – thumbs up දුන්නා.)

R code

```
# Build list of item->rating per user
by_user <- split(train, train$user)
user_items <- lapply(by_user, function(d) {
  setNames(d$rating, d$item)
})

head(by_user)
head(user_items)
mode(user_items)     # "list"
```

- `split()` train data, **එක් user කෙනෙකුට** එක කුඩා data frame එකක් ලෙස බෙදනවා (`by_user`).

- **Apply family** එකේ functions: `apply`, `lapply`, `sapply`, `mapply`, `tapply`. ඒවා **data source** එකක්, (`apply` සඳහා) **දිශාවක්** (row අනුව හෝ column අනුව), සහ එක් එක් කොටසට යොදන **function** එකක් ගන්නවා.
- `lapply` **list** එකක් දෙනවා. හැම user කෙනෙකුටම නම් සහිත vector එකක් හදනවා: අගයන් = ratings, නම් = item IDs (`setNames(d$rating, d$item)`). Sparse utility matrix එක ගබඩා කරන්න memory වලට හිතකර ක්‍රමයක් – blank නොවන අගයන් විතරයි ගබඩා කරන්නේ.
- List එකක් matrix එකක් වගේ index කරන්න බැහැ; පළමු සාමාජිකයා (user 1 ගේ ratings) ගන්න `user_items[["1"]]` හෝ `user_items$`1`` පාවිච්චි කරන්න.
- එකවර බලන්න බැරි data සාරාංශවලින් explore කරන්න, උදා: `summary(ratings$item)`, `max(ratings$item)`. (MovieLens 100K වල users 943 ක් සහ movies 1,682 ක් ඉන්නවා.)

6.4 Baseline model (user bias සහ item bias)

R code

```
mu <- mean(train$rating)                                    # overall mean rating

user_bias <- tapply(train$rating - mu, train$user, mean)   # how much each user
item_bias <- tapply(train$rating - mu, train$item, mean)   # / item deviates from mu

predict_baseline <- function(u, i) {
  bu <- if (!is.na(user_bias[as.character(u)])) user_bias[as.character(u)] else 0
  bi <- if (!is.na(item_bias[as.character(i)])) item_bias[as.character(i)] else 0
  mu + bu + bi
}

# RMSE on test
pred_base <- mapply(predict_baseline, test$user, test$item)
rmse <- function(y, yhat) sqrt(mean((y - yhat)^2))
rmse(test$rating, pred_base)
```

- **mu** = සියලු training ratings වල සාමාන්‍යය.
- **User bias**: හැම user කෙනෙකුටම (rating − mu) හි සාමාන්‍යය – මේ user සාමාන්‍යයෙන් සාමාන්‍යයට වඩා ඉහළින්ද පහළින්ද rate කරන්නේ?
- **Item bias**: හැම item එකකටම (rating − mu) හි සාමාන්‍යය – මේ movie එකට සාමාන්‍යයෙන් සාමාන්‍යයට වඩා ඉහළ හෝ පහළ ratings ලැබෙනවද?
- Baseline prediction = **mu + user bias + item bias** (user හෝ item නොදන්නා නම් bias = 0). මේක Netflix Challenge හි සරල "සාමාන්‍ය" algorithm එකට ගොඩක් සමානයි.
- `mapply` test set එකේ හැම (user, item) යුගලයකටම function එක යොදනවා. ඊට පස්සේ test set එකේ **RMSE** ගණනය කරනවා. පන්තියේදී RMSE එක **0.95** පමණ ආවා.
- පහත CF functions මේ baseline එකට ආපසු වැටෙනවා (fall back), ඒ නිසා **baseline එක මුලින්ම define කරන්න** (පන්තියේදී "could not find function predict_baseline" error එක ආවේ CF code එක මේ කොටසට කලින් run කළ නිසා; `mu` ත් මුලින් define කරන්න වුණා).

6.5 Cosine similarity සහ user-based CF

User-based CF	Item-based CF
item i දැනටමත් rate කරපු, **ඔයාට වඩාත්ම සමාන users** හොයනවා.	ඔයා දැනටමත් rate කරපු items අතරින් **item i ට වඩාත්ම සමාන items** හොයනවා.
Prediction = item i ට **ඔවුන්** දුන් ratings වල weighted average එක	Prediction = ඒ items වලට **ඔයා** දුන් ratings වල weighted average එක

R code

```
# Cosine similarity on co-rated items
cosine_sim <- function(u_vec, v_vec) {
  common <- intersect(names(u_vec), names(v_vec))
  if (length(common) < 5) return(NA_real_)  # require overlap
  u <- u_vec[common]; v <- v_vec[common]
  sum(u*v) / (sqrt(sum(u^2)) * sqrt(sum(v^2)))
}

predict_user_cf <- function(u, i, k = 30) {
  u <- as.character(u); i <- as.character(i)
  if (is.null(user_items[[u]])) return(mu)

  # Find candidate neighbors who rated item i
  neighbors <- names(Filter(function(x) i %in% names(x), user_items))
  neighbors <- setdiff(neighbors, u)
  if (length(neighbors) == 0) return(predict_baseline(u, i))

  sims <- sapply(neighbors, function(v) cosine_sim(user_items[[u]], user_items[[v]]))
  sims <- sims[!is.na(sims)]
  if (length(sims) == 0) return(predict_baseline(u, i))

  top <- head(sort(sims, decreasing = TRUE), k)
  # weighted sum of neighbors' ratings for item i
  r_vi <- sapply(names(top), function(v) user_items[[v]][[i]])
  sum(top * r_vi) / sum(abs(top))
}

pred_ucf <- mapply(predict_user_cf, test$user, test$item)
rmse(test$rating, pred_ucf)
```

- **cosine_sim**: මෙතැන `intersect()` පාවිච්චි කරන්නේ **නම්** (item IDs) මත, ඒ නිසා users **දෙදෙනාම rate කරපු** items (co-rated items) නිවැරදිව හොයාගන්නවා. පොදු items **5 ට අඩු** නම් similarity එක විශ්වාස කරන්නේ නැහැ (NA). ඊට පස්සේ අපි ඉගෙනගත්ත cosine සූත්‍රය යොදනවා.
- **predict_user_cf(u, i, k = 30)** – user u, item i ට දෙන rating එක predict කරන්න:
- 1. u නොදන්නා කෙනෙක් නම්, සමස්ත සාමාන්‍යය mu දෙන්න.
- 2. **item i rate කරපු** අනෙක් සියලු users (neighbours) හොයන්න (`Filter` item i අඩංගු lists විතරක් තියාගන්නවා; `setdiff` u ඉවත් කරනවා).
- 3. u සහ එක් එක් neighbour අතර cosine similarity ගණනය කරන්න; NA ඉවත් කරන්න. කිසිවක් ඉතිරි නැත්නම් baseline එක පාවිච්චි කරන්න.
- 4. වඩාත්ම සමාන **top k = 30** neighbours තියාගන්න.
- 5. Prediction = item i ට ඔවුන් දුන් ratings වල **weighted average**: Σ(sim × rating) / Σ|sim|.

6.6 Item-based CF

R code

```
by_item <- split(train, train$item)
item_users <- lapply(by_item, function(d) {
  setNames(d$rating, d$user)
})

predict_item_cf <- function(u, i, k = 30) {
  u <- as.character(u); i <- as.character(i)
  if (is.null(item_users[[i]])) return(predict_baseline(u, i))

  # Items that user u has rated
  u_ratings <- user_items[[u]]
  if (is.null(u_ratings)) return(mu)

  rated_items <- names(u_ratings)
  rated_items <- setdiff(rated_items, i)
  if (length(rated_items) == 0) return(predict_baseline(u, i))

  sims <- sapply(rated_items, function(j) cosine_sim(item_users[[i]], item_users[[j]]))
  sims <- sims[!is.na(sims)]
  if (length(sims) == 0) return(predict_baseline(u, i))

  top <- head(sort(sims, decreasing = TRUE), k)
  r_uj <- u_ratings[names(top)]
  sum(top * r_uj) / sum(abs(top))
}

pred_icf <- mapply(predict_item_cf, test$user, test$item)
rmse(test$rating, pred_icf)
```

- මේ වතාවේ බෙදන්නේ **item** අනුව: හැම movie එකකටම ලැබුණු ratings වල නම් සහිත vector එකක් (නම් = user IDs). මේවා utility matrix එකේ **columns** – **ද්විත්වය (duality)** මතක් කරගන්න: එකම similarity අදහස් rows (users) වලට හෝ columns (items) වලට යොදන්න පුළුවන්.
- User u, item i ට දෙන rating එක predict කරන්න: **u දැනටමත් rate කරපු items** බලලා, ඒ අතරින් **item i ට වඩාත්ම සමාන** ඒවා හොයලා (item columns අතර cosine similarity), top 30 තියාගෙන, ඒ items වලට **u දුන් ratings** වල weighted average එක ගන්න.

6.7 User කෙනෙකුට Top-N recommendations

R code

```
# Recommend items the user hasn't rated, using item-CF predictions.
recommend_topN <- function(user_id, N = 10) {
  u <- as.character(user_id)
  seen <- if (!is.null(user_items[[u]])) names(user_items[[u]]) else character()
  all_items <- names(item_users)
  candidates <- setdiff(all_items, seen)

  scores <- sapply(candidates, function(i) predict_item_cf(u, i, k = 30))
  topN <- head(sort(scores, decreasing = TRUE), N)
  data.frame(item = names(topN), score = as.numeric(topN), row.names = NULL)
}

recommend_topN(1, 10)
```

- Recommender එකක **ප්‍රධාන අදහස** මේකයි: user කෙනෙකුට ඔහු **තවම නොබලපු** items (candidates) හොයලා, එක් එක් එකට score එකක් predict කරලා, **top N** (මෙතැන 10) පෙන්වන එක.
- පන්තියේදී `recommend_topN(1, 10)` මුලින්ම error එකක් දුන්නා (“item_users not found”), මොකද item-based කොටස තවම run කරලා තිබුණේ නැහැ – ඒක run කළාට පස්සේ වැඩ කළා, **user**

1 ට top 10 items (movie IDs) ඒවායේ predicted scores එක්ක ආවා.

- Records 100,000 ක් තියෙන නිසා run වෙන්න ගොඩක් වෙලා යනවා. **ඉඟිය:** function එක ඇතුළේ `print()` එකක් දාලා අතරමැදි output පෙන්වන්න, එතකොට පැය ගණන් හිස් window එකක් බලන් ඉන්නේ නැතිව progress එක බලාගන්න පුළුවන්.

7. නිවේදන සහ homework

7.1 නිවේදන

- **ලබන සඳුදා lab session එකක්** (tutor ට දැනුම් දීලා තියෙනවා). Lab computers වැඩ කරනවද කියලා batch representative **කලින්ම** tutor එක්ක පරීක්ෂා කරන්න ඕනේ. වැඩ කරන්නේ නැත්නම් කවුරුවත් එතැනට ගිහින් කාලය නාස්ති නොකරන්න දේශකතුමාට කලින්ම දන්වන්න. Lab එක නැත්නම් ගෙදර / boarding / hostel එකේ laptop එකෙන් කරන්න – නමුත් lab එකට යන එක නිර්දේශ කරනවා.
- **ලබන සතියේ සිට computer එකක් ඉස්සරහා ඉන්න** (laptop එකක් ගේන්න හෝ බෙදාගන්න). ඔයාලා models train කරනවා – **support vector machines, neural networks** – code එක බලන් ඉන්නවා විතරක් මදි, run කරන්න ඕනේ. අවධානය තියාගන්න සමහර programs වල ශිෂ්‍යයන්ට video on කරන්න කියනවා.
- මේ course එකේ **lab hours** සැලකිය යුතු ප්‍රමාණයක් තියෙනවා, ඒ නිසා hands-on පුහුණුව ගොඩක් වැදගත්.
- **ලබන දේශනය:** parametric සහ non-parametric **supervised learning**, සහ **support vector machines (SVM)** හැඳින්වීම.
- Semester දින: whiteboard එකේ තියෙන්නේ **semester එකේ අවසාන දිනය: 20/03/2026**. (පටිගත කිරීමේදී දේශකතුමා මතක හැටියට අවසාන දිනය පෙබරවාරි අග පමණ සහ විභාග මාර්තු දෙවන සතියේ පමණ කියලත් කිව්වා – පීඨයේ නිල නිවේදනය අනුගමනය කරන්න.)
- පැමිණීම: ඔයාගේ නම සහ registration number එක (සහ ඉල්ලපු R code එක) chat එකේ එවන්න.
- **Homework දැන්ම කරන්න** – අන්තිම මොහොත වෙනකම් ඉන්නවා නම්, පැය 60 කට වඩා recordings බලන්න වෙනවා, ඒක ප්‍රායෝගික නැහැ.

7.2 Homework ලැයිස්තුව

#	කාර්යය	සටහනේ කොතැනද
1	Exercise 9.2.2 (a): computers තුනේ vectors normalize කරන්න	1.5 කොටස
2	Exercise 9.2.3 (a), (b): user ගේ ratings normalize කරලා user profile එක හදන්න	1.5 කොටස
3	HW2: සියලු යුගල සඳහා Jaccard සහ cosine similarity (Figure 9.4)	2.8 කොටස
4	Figure 9.4 හි matrix එක R වලින් normalize කරන්න	2.6 කොටස
5	Normalize කළාට පස්සේ A සහ B අතර cosine distance – (I) අතින්, (II) R වලින්	2.7 කොටස
6	Exercise 9.3.1 (a)–(f), විශේෂයෙන් part (c) R වලින්	3 කොටස
7	**ඔයාගේම Jaccard distance function එක R වලින් ලියන්න (LMS, සතියකින්)**	3.1 කොටස
8	MovieLens recommender code එක තනියම run කරන්න; Netflix-challenge documentary එකක් බලන්න	5, 6 කොටස්

8. සාරාංශය

සංකල්පය	සරල වචනවලින්
Utility matrix	Users × items ratings වගුව; බොහෝ දුරට හිස් (sparse). අරමුණ: blanks predict කිරීම.
Content-based	User කැමති වුණ items වලට සමාන **ගුණාංග** තියෙන items recommend කිරීම.
Collaborative filtering	**සමාන users** (rows) හෝ **සමාන items** (columns) හොයලා, සමාන users කැමති දේ recommend කිරීම.
Jaccard	Items **කට්ටල** මත $\|A \cap B\| / \|A \cup B\|$; rating අගයන් නොසලකා හරිනවා. මිලදී ගැනීම් (0/1) data වලට හොඳයි.
Cosine	Dot product / (දිග); rating අගයන් පාවිච්චි කරනවා; blanks = 0. Cosine විශාල = කෝණය කුඩා = වඩා සමානයි.
Rounding	3, 4, 5 → 1; 1, 2 → blank/0. Jaccard සහ cosine වලට අර්ථවත් උත්තර දෙන්න උදව් වෙනවා.
Normalizing	එක් එක් user ගේ සාමාන්‍ය rating එක අඩු කිරීම: කැමති → ධන, අකමැති → ඍණ. ප්‍රතිවිරුද්ධ users ලට ප්‍රතිවිරුද්ධ vectors.
UV-decomposition	M (n × m) ≈ U (n × d) × V (d × m); d = සැඟවුණු features කිහිපයක්; UV මඟින් blanks පුරවනවා.
RMSE	√(blank නොවන අගයන්වල වර්ග කළ දෝෂවල සාමාන්‍යය). අඩු වෙන තරමට හොඳයි. Netflix Challenge හි පාවිච්චි කළා.
Netflix Challenge	CineMatch (RMSE ≈ 0.95) 10% කින් පරාජය කරන්න $1M; 2009 දී algorithms එකතුවකින් දිනුවා.
R recommender	train/test බෙදීම → user/item rating lists → පොදු items මත cosine → top-k neighbours ගේ weighted average → RMSE → top-N.

උපග්‍රන්ථය: පන්තියේ සම්පූර්ණ R code එක (Day 6)

මේ පන්තියේදී බෙදාගත්ත code එකමයි, comments කිහිපයක් විතරක් එකතු කළා. Figure 9.8 matrix එකේ data ඇතුළත් කිරීමේ නිවැරදි කිරීම (3.1 කොටස) සහ කොටස් run කරන පිළිවෙළ (මුලින් baseline, ඊට පස්සේ CF, ඊට පස්සේ item-based, අන්තිමට top-N) මතක තියාගන්න.

Day 6 – පන්තියේ R code

```
# Day 6
#   a b c d e f g h
# A 4 5   5 1   3 2
# B   3 4 3 1 2 1
# C 2   1 3   4 5 3

# store this in a matrix/data frame

ratings = matrix(c(
  4, 5, NA, 5, 1, NA, 3, 2,
  3, 4, 3, 1, 2, 1, NA, NA,
  2, 1, 3, NA, 4, 5, 3, NA
), nrow = 3, byrow = TRUE)
# NOTE: rows B and C above do not match the table (see Section 3.1).
# Correct: NA,3,4,3,1,2,1,NA  and  2,NA,1,3,NA,4,5,3

ratings
colnames(ratings) = c(letters[1:8])

letters
LETTERS

ratings
rownames(ratings) = LETTERS[1:3]
ratings

# Treat ratings of 3, 4, and 5 as 1 and 1, 2, and blank as 0.

binaryRatings = ifelse(ratings>=3,1,0)
binaryRatings

## replace missing with a zero??

binaryRatings[is.na(binaryRatings)] = 0
binaryRatings

# Compute the Jaccard distance between each pair of users.

## JD = 1 - intersection/union
## pairs => AB, AC and BC

# HW: due in a week
JaccardDist = function(UM,A,B){
  intersect(UM[A,], UM[B,])/union(UM[A,], UM[B,])
}

intersect(binaryRatings[1,], binaryRatings[2,])
intersect(binaryRatings[1,], binaryRatings[2,])/union(binaryRatings[A,], binaryRatings[B,])
JaccardDist(binaryRatings,A=1,B=2)

######
# Create a workspace
dir.create("ml-100k", showWarnings = FALSE)

# Download MovieLens 100K
zip_url <- "https://files.grouplens.org/datasets/movielens/ml-100k.zip"
zip_file <- "ml-100k.zip"
download.file(zip_url, zip_file, mode = "wb")
unzip(zip_file, exdir = ".")

# Load ratings (u.data: user_id, item_id, rating, timestamp)
ratings <- read.table("ml-100k/u.data", sep = "\t",
                      col.names = c("user", "item", "rating", "ts"))
head(ratings)

# split the data into train and test

set.seed(42)
```

```
split_by_user <- function(df, test_frac = 0.2) {
  idx_test <- unlist(tapply(seq_len(nrow(df)), df$user, function(ix) {
    n <- length(ix)
    n_test <- max(1, floor(n * test_frac))
    sample(ix, n_test)
  }))
  test <- df[idx_test, ]
  train <- df[-idx_test, ]
  list(train = train, test = test)
}

spl <- split_by_user(ratings, 0.2)
train <- spl$train
test  <- spl$test

dim(ratings)
# 100000 rows and 4 cols

dim(train)
#80367     4
dim(test)
#19633     4

# User-based Collaborative Filtering (from scratch)
# Steps
# 1. Create a user-item matrix (sparse-like approach)
# 2. Compute similarity between users (cosine / Pearson)
# 3. Predict from neighbors' ratings

# Build list of item->rating per user
by_user <- split(train, train$user)
user_items <- lapply(by_user, function(d) {
  setNames(d$rating, d$item)
})

head(by_user)
head(user_items)

mode(user_items)

####################
mu <- mean(train$rating)

user_bias <- tapply(train$rating - mu, train$user, mean)
item_bias <- tapply(train$rating - mu, train$item, mean)

predict_baseline <- function(u, i) {
  bu <- if (!is.na(user_bias[as.character(u)])) user_bias[as.character(u)] else 0
  bi <- if (!is.na(item_bias[as.character(i)])) item_bias[as.character(i)] else 0
  mu + bu + bi
}

# RMSE on test
pred_base <- mapply(predict_baseline, test$user, test$item)
rmse <- function(y, yhat) sqrt(mean((y - yhat)^2))
rmse(test$rating, pred_base)

# (The rest of the code - cosine_sim, predict_user_cf, by_item / item_users,
#  predict_item_cf and recommend_topN - is exactly as shown in
#  Sections 6.5, 6.6 and 6.7.)
```